

Chapitre 1 : Technologies de la programmation web

Que veut-on dire par « Technologies de la programmation Web »?

Les **technologies de programmation web** sont les outils et langages utilisés pour créer des sites et applications web. Elles évoluent constamment.

Technologies traditionnelles :

- **HTML** : structure et contenu des pages web.
- **CSS** : style et mise en forme des pages.
- **JavaScript** : interactivité et dynamique côté client (ou serveur).
- **PHP** : scripts côté serveur pour sites dynamiques.
- **ASP.NET** : framework Microsoft pour applications web robustes.
- **Java** : utilisé côté serveur avec Servlets/JSP.
- **Ruby on Rails** : framework Ruby pour développement rapide.
- **Python** : langage polyvalent avec frameworks web (Django, Flask).
- **SQL** : manipulation et requêtes de bases de données relationnelles.

Technologies modernes :

- **Front-end (interfaces utilisateur) :**
 - **React, Angular, Vue.js** : bibliothèques/frameworks JavaScript pour créer des interfaces interactives et des applications web monopage.
 - **Sass** : extension CSS avec variables et fonctionnalités avancées.
 - **Webpack** : outil pour gérer et optimiser le code des applications complexes.
- **Back-end (serveur et bases de données) :**
 - **Node.js** : exécution JavaScript côté serveur.
 - **Express.js** : framework Node.js pour créer des applications serveur.
 - **GraphQL** : langage de requêtes pour API, récupérant exactement les données demandées.
 - **MongoDB** : base de données NoSQL orientée documents JSON.
 - **Firebase** : plateforme Google pour bases de données en temps réel, auth, stockage, etc
- **Outils de déploiement et conteneurisation :**
 - **Docker** : conteneurs légers pour exécuter des applications avec toutes leurs dépendances sur n'importe quel système.

Motivation :

Salaire selon l'expérience :

- **Débutant** : 35 000 – 47 000 € / an (2 900 – 3 900 € / mois)
- **Confirmé (3–6 ans)** : 48 000 – 60 000 € / an
- **Senior** : > 60 000 € / an
- **Back-end** : débutant 40 000 – 47 000 € / a
- **Full stack** :
 - Débutant : 38 000 – 42 000 € / an
 - 1–3 ans : 42 000 – 50 000 € / an

- 3 ans : 50 000 – 65 000 € / an
- 6 ans : > 65 000 € / an
- **Freelance** : junior ~300 €/jour, senior ~550 €/jour

Pourquoi exercer ce métier ?

- Forte demande => salaire attractif
- Technologies en évolution => apprentissage constant
- Télétravail de plus en plus répandu

Généralités sur le développement web

Le développement web :

- Ensemble des tâches pour créer et gérer un site web : conception, codage, gestion du contenu, maintenance.
- **Tests** : vérifient le bon fonctionnement et l'exactitude du contenu.
- **Mission principale du développeur** : créer des sites **attrayants, accessibles et optimisés pour le référencement**.

Le frontend développeur :travaille sur ce que l'internaute voit

- Travaille sur l'**interface visible** par l'utilisateur.
- **Mission** : créer une expérience utilisateur fluide et agréable.
- **Tâches principales** : mise en page, boutons, menus, formulaires, liens, animations, interactions, responsive design.
- **Collaboration** : avec webdesigner et expert UX.

Un expert UX (User Experience) :

- Spécialiste de la **conception et de l'amélioration de l'expérience utilisateur**.
- **Objectif** : optimiser la satisfaction, l'efficacité, la facilité d'utilisation et la valeur perçue par l'utilisateur.
- **Domaines d'intervention** : applications mobiles, sites web, logiciels, produits matériels, systèmes embarqués, etc.
- **Rôle** : s'assurer que le produit répond efficacement et intuitivement aux besoins des utilisateurs.

Les langage utilisés en frontend :

- **HTML** : structure et organise le contenu (texte, images, polices, couleurs).
- **CSS** : met en forme et stylise les pages (layout, couleurs, design).
- **JavaScript** : ajoute de l'**interactivité** (menus, formulaires, animations).

Frameworks Front-end :

- Squelettes de code pré-écrit pour **accélérer le développement**.
- Exemples : **Bootstrap, React.js, Angular.js**.

Bibliothèques Front-end :

- Paquets de code réutilisables à intégrer dans le projet.
- Exemples : **jQuery**, **LESS**.

Le backend développeur :travaille sur ce que l'internaute ne voit pas

- Travaille sur **ce que l'utilisateur ne voit pas** (serveur et logique interne).
- **Mission** : mettre en place les fonctionnalités et gérer la transmission des données aux utilisateurs.
- **Tâches principales** :
 - Communication serveur ↔ application via **API**
 - Gestion de la **logique back-end**
 - Respect des **politiques de sécurité et d'accessibilité**

Les langage de programmation utilisés en frontend :

- **PHP** : script côté serveur pour sites web dynamiques.
- **Java** : langage polyvalent, fonctionne sur toutes les plateformes compatibles.
- **Python** : puissant et populaire, idéal pour scripts et applications multi-plateformes.
- **C++** : utilisé pour la programmation concurrente et performante.

Les frameworks et bibliotheques back-end :

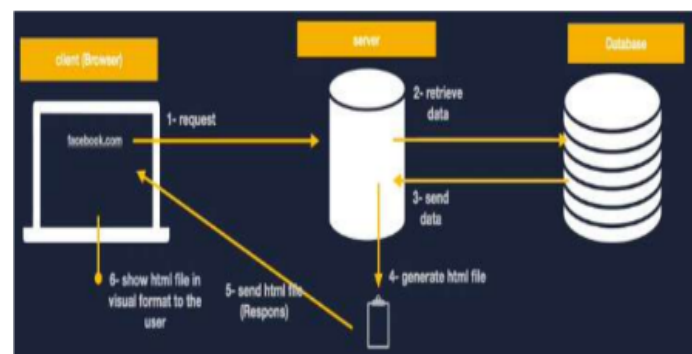
- Permettent **d'accélérer le développement** et de structurer le code.
- Exemples :
 - **Node.js** : environnement d'exécution JavaScript côté serveur
 - **Laravel, Symfony** : frameworks PHP
 - **Django** : framework Python
 - **Ruby on Rails** : framework Ruby

Fonctionnement du web : Front-end ↔ Back-end

Objectif : relier le client (Front-end) au serveur (Back-end).

Intégration directe : le backend génère le **HTML, vues et templates** et les envoie directement au navigateur.

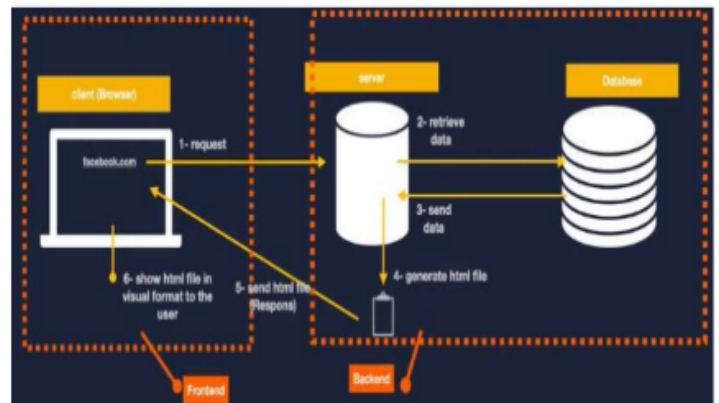
- Typique des applications **monolithiques**.
- Moins **modulaire** et **évolutif** qu'avec une API



Fonctionnement du web : comment séparer le back end du front end ?

Parties du système :

- **Front-end** : visible par l'utilisateur dans le navigateur (HTML, images, boutons...). Exemple : page Facebook.
- **Back-end** : serveur + base de données.
 - Le serveur exécute le code (PHP, Node.js, Python...).
 - La base de données stocke les informations (comptes, posts, messages...).



Fonctionnement en 6 étapes :

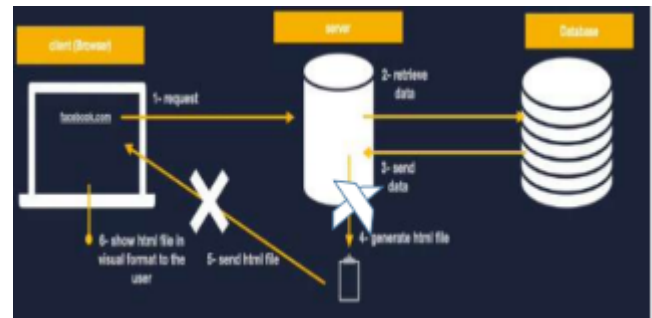
1. Le **navigateur** envoie une requête (request) au serveur.
2. Le **serveur** demande les données nécessaires à la base de données.
3. La **base de données** renvoie les informations au serveur.
4. Le **serveur** génère la page HTML à partir des données.
5. Le **serveur** envoie la page HTML au navigateur (response).
6. Le **navigateur** affiche la page (interface visuelle : textes, images, boutons...).

Fonctionnement du web : liens entre le back end et le front end

- **Problème illustré** : le serveur ne génère pas ou n'envoie pas la page HTML.
- **Conséquence** : le navigateur ne reçoit rien → page blanche, erreur ou timeout.

Étapes :

1. Le navigateur envoie une requête au serveur.
2. Le serveur demande des données à la base.
3. La base renvoie les données au serveur.
4. Le serveur **ne génère pas la page HTML** (étape bloquée).
5. Le serveur **n'envoie rien au navigateur** (étape bloquée).
6. Le navigateur **n'affiche rien**.



Causes possibles :

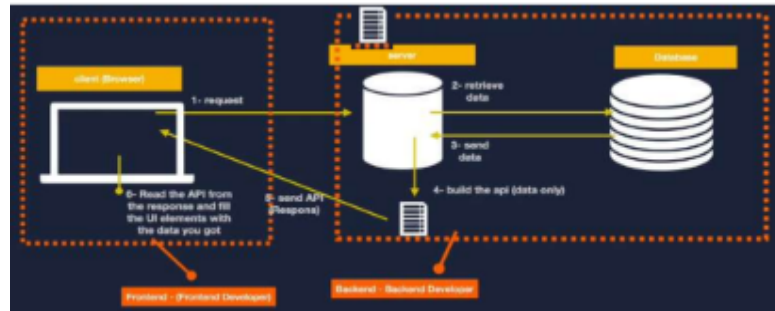
- Bug dans le code serveur.
- Problème interne ou crash.
- Mauvais format des données de la base.
- Route back-end renvoie rien ou erreur.

Solution classique : introduire une **API** comme couche intermédiaire pour gérer les échanges Front-end / Back-end et éviter de dépendre de la génération directe du HTML.

Fonctionnement du web : liens entre le back end et le front end(Suite)

Étapes :

1. Le **client (navigateur)** envoie une requête (API).
2. Le **serveur** récupère les données auprès de la base.
3. La **base de données** renvoie les informations au serveur.
4. Le serveur **prépare une réponse JSON** (données brutes), pas de page HTML.
5. Le serveur **envoie la réponse API** au Front-end.
6. Le **Front-end** (React, Vue, Angular...) lit les données et **construit l'interface visuelle** : HTML, textes, images, mise en forme.



Avant (pages classiques) : le Back-end générait et envoyait la page HTML complète.

Maintenant (SPA, apps modernes) :

- **Back-end** : récupère les données et renvoie JSON
- **Front-end** : reçoit les données et construit la page

Avantages de ce modèle :

- Plus **rapide** et **flexible**
- Compatible avec les **applications mobiles**
- Permet l'utilisation de **React, Vue, Angular**
- Séparation claire **Front-end / Back-end**

Fonctionnement du web : liens entre le back end et le front end (Suite)

Rôle de l'API :

- **Cache le Back-end** : le Front-end ne voit pas ce qui se passe côté serveur.
- Le Front-end **interagit uniquement avec l'API**, qui fournit les données nécessaires.
- L'utilisateur final ne voit que l'interface construite par le Front-end, pas le traitement Back-end.

C'est quoi une api?

API (Application Programming Interface) :

- Ensemble de **règles, protocoles et outils** permettant à différents logiciels de **communiquer**.
- Offre une **interface aux développeurs** pour accéder aux fonctionnalités d'un logiciel ou service.

API Web :

- Permet la communication entre applications via Internet.
- Utilise des protocoles comme **HTTP**.
- Retourne des données souvent au format **JSON** ou **XML**.

API REST :

- Méthode courante pour relier **Back-end et Front-end**.
- Le Back-end expose des **endpoints** via HTTP.
- Utilise des méthodes comme **GET, POST, PUT, DELETE**.
- Le Front-end envoie des requêtes et reçoit des **réponses JSON ou XML**.

Bibliothèques et frameworks : De nombreux frameworks frontend, tels que Les frameworks Front-end (**React, Angular, Vue.js**) offrent des outils pour faciliter les échanges avec le Back-end.

Exemples :

- **Axios** : bibliothèque pour envoyer des requêtes HTTP facilement.
- **fetch()** : fonction native JavaScript pour interagir avec le Back-end.

Requêtes HTTP :

- Permettent l'**interaction Front-end ↔ Back-end**.
- Utilisées pour **demandeur des ressources** : pages web, images, fichiers, données JSON depuis une API.

Exemple d'API HTTP : JSONPlaceholder

- Service en ligne **REST gratuit** pour tester du code ou des prototypes.
- **URL** : <https://jsonplaceholder.typicode.com/>
- Fournit des **routes / endpoints** correspondant à différentes ressources (ex. utilisateurs, posts, commentaires).
- Permet d'envoyer des **requêtes HTTP** (GET, POST, etc.) et de recevoir des **données JSON**.

JSONPlaceholder – Routes / Endpoints et Ressources

- Fournit plusieurs **endpoints** pour accéder à des **ressources de test** via requêtes HTTP.
- **Exemples de ressources disponibles** :
 - **/users** → informations sur les utilisateurs
 - **/posts** → publications (posts)
 - **/comments** → commentaires sur les posts
 - **/albums** → albums photos
 - **/photos** → photos
 - **/todos** → tâches à faire

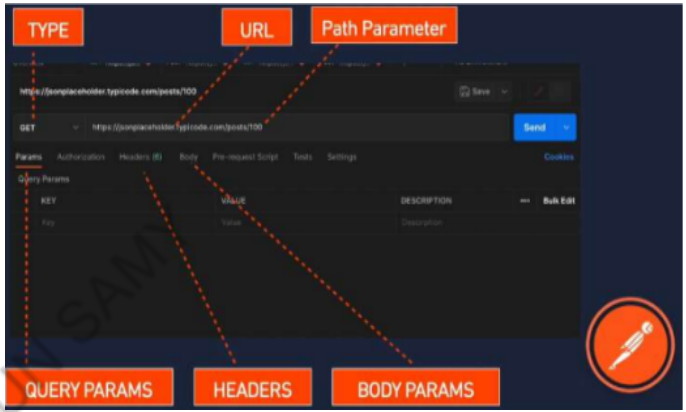
- Chaque endpoint retourne des **données JSON** utilisables pour tester le Front-end ou les API.

Outils de test et d'interaction d'API :

1. **Postman** :
 - Permet de **créer, envoyer et gérer des requêtes HTTP**.
 - Collecte et organise les **réponses**.
 - Offre des fonctionnalités de **tests automatisés** et de **documentation d'API**.
2. **Insomnia** :
 - Client RESTful pour **tester et gérer des requêtes HTTP**.
 - Inclut des outils pour **gestion d'environnement** et **génération de code**.

Interface de Postman :

Parties d'une requête API :

1. **Type** : type de requête HTTP
 - **GET** → récupérer des données
 - **POST** → envoyer des données
 - **PUT** → mettre à jour
 - **DELETE** → supprimer
 2. **URL** : adresse de l'API / backend
 - Exemple :
- 
- <https://jsonplaceholder.typicode.com/posts/100>
3. **Path Parameter** : identifie une ressource précise dans l'URL
 - Exemple : [/posts/100](#) → post n°100
 - Autres exemples : [/users/12](#), [/products/55](#)
 4. **Query Params** : options pour filtrer, trier ou limiter les résultats
 - Exemple : [?limit=10&sort=asc](#)
 5. **Headers** : informations supplémentaires envoyées avec la requête
 - Exemple : **Authorization:** [Bearer token](#), **Content-Type:** [application/json](#)
 - Utilité : authentification, format des données, métadonnées
 6. **Body Params** : données envoyées dans le corps de la requête (POST, PUT, PATCH)
 - Exemple : `{"title": "Hello", "body": "World"}`
- Non utilisé pour GET

Résumé tableau :

Élément	Rôle
Type	Action (GET/POST/PUT/DELETE)
URL	Adresse du serveur
Path Param	ID de la ressource ciblée

Query Params	Filtrer / trier / limiter les résultats
Headers	Infos techniques (token, format...)
Body Params	Données envoyées au backend

Structure générale d'une requête HTTP

Structure d'une requête HTTP (Postman, Insomnia, Hoppscotch...) :

REQUEST	
URL	Ex: https://jsonplaceholder.typicode.com/todos/1
TYPE	GET / POST / PUT / PATCH / DELETE
PARAMS	PATH PARAMS - QUERY PARAMS - BODY PARAMS
HEADERS	Some information about the request

1. **Request** : titre ou nom de la requête (ex. "Récupérer la liste des tâches")
2. **URL** : adresse complète de l'API
 - Exemple : <https://jsonplaceholder.typicode.com/todos/1>
3. **Type / Méthode HTTP** : verbe indiquant l'action
 - GET** → récupérer des données
 - POST** → créer des données
 - PUT / PATCH** → mettre à jour des données
 - DELETE** → supprimer des données
4. **Params** : paramètres ajoutés à la requête
 - Path Params** : variables dans l'URL (ex. `/users/{id}`)
 - Query Params** : après `?` dans l'URL (ex. `?page=2&size=10`)
 - Body Params** : données envoyées dans le corps (POST/PUT/PATCH, souvent en JSON)
5. **Headers** : informations supplémentaires
 - Exemples :
 - Content-Type: application/json
 - Authorization: Bearer token
 - Accept: application/json

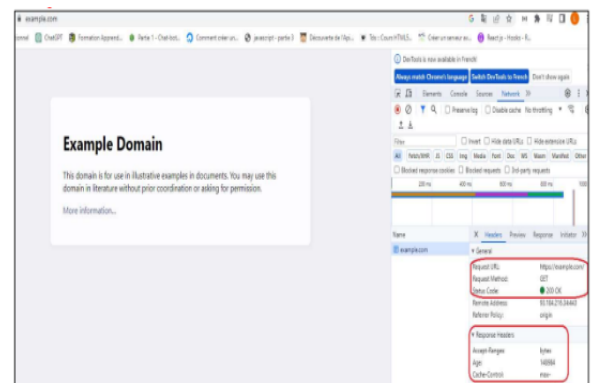
Résumé simple : l'écran d'un outil API montre où remplir **URL**, **type**, **params** et **headers** pour construire et envoyer une requête REST.

Exemple 1 – Requête HTTP via navigateur (Chrome DevTools, onglet Network)

- **Page web** : <https://example.com> (site d'exemple, juste pour démonstration).
- **Onglet Network** : montre la requête réellement envoyée.

Détails de la requête :

- **Name** : example.com → nom de la ressource demandée
- **Status** : 200 OK → serveur a répondu correctement



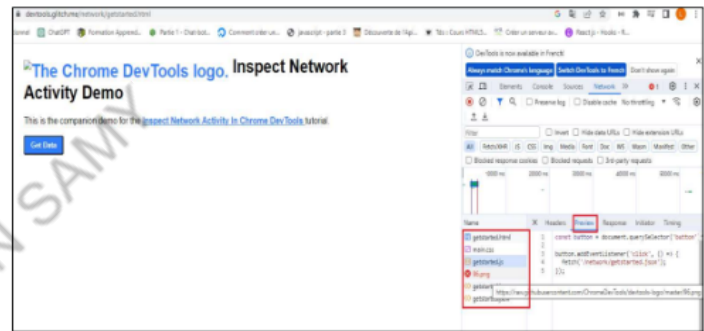
- **Method** : GET → type de requête
- **Request URL** : <https://example.com/> → URL complète appelée
- **Remote Address** : IP et port du serveur (ex. 93.184.216.34:443)
- **Referrer Policy** : règle de sécurité du navigateur

Résumé simple :

- Tu configures une requête (URL + méthode + éventuellement headers/params)
- Le navigateur l'exécute automatiquement
- Résultat visible dans Network (statut, méthode, URL, réponse)
- **Principe identique** à l'envoi d'une requête via Postman ou un autre client API.

Exemple 2 – Cycle complet d'une requête HTTP dynamique (AJAX / Fetch)

- **Page web**
: <https://googlechromedevelopers.googleblog.com/.../inspect-network-activity.html>
 - Démo interactive avec un bouton "Get Data"
- **Onglet Network (DevTools)** : montre toutes les requêtes déclenchées
 - **getstarted** → Status 200 → fichier HTML de démarrage
 - **getstarted.js** → Status 200 → code JavaScript de la démo
 - **getstarted.json** → Status 200 → requête GET déclenchée par JS (Fetch / XHR)



```
{\"title\": \"Inspect Network Activity\", \"description\": \"This is the companion demo...\"}
```

Pourquoi cette démo est utile :

- Montre le **cycle complet** :
 1. Chargement initial de la page (HTML, JS, CSS)
 2. Interaction utilisateur (clic sur le bouton)
 3. Requête dynamique GET vers un fichier JSON
 4. Résultat visible en temps réel dans Network (nom, méthode, statut, type, taille, temps)

Résumé simple :

- Illustre comment une **page web récupère des données dynamiquement**.
- Même principe utilisé par **React, Vue, Angular** ou toute app consommant une API

Le contenu d'une requête HTTP

1/URL (Uniform Resource Locator) :

- Sert à **localiser une ressource sur un serveur**.
- Composants principaux :
 1. **Protocole** : [http://](#) ou [https://](#)
 2. **Nom de domaine** : exemple [www.example.com](#)
 3. **Path Params / Chemin** : [/posts/1](#) → identifie une ressource spécifique
 4. **Query Params** (optionnel) : après **?** pour filtrer, trier ou limiter ([?param1=value1¶m2=value2](#))

Exemples :

- [https://www.example.com/page1?param1=value1¶m2=value2](#) → avec query params
- [https://jsonplaceholder.typicode.com/posts/1](#) → path param (1 identifie un post précis)

2. Champ d'en-tête de la requête (HEADERS)

- Fournit des **informations supplémentaires** sur la requête HTTP.
- Format : **key: value** (souvent en JSON ou texte).

Exemples d'usage :

- **Type de navigateur** : [User-Agent: Mozilla/5.0 \(Windows NT 10.0; Win64; x64\) AppleWebKit/537.36 \(KHTML, like Gecko\) Chrome/80.0.3987.132 Safari/537.36](#)
- **Type de contenu accepté** : [Accept-Charset: utf-8](#)
- **Authentification** : [Authorization: Bearer <token>](#)
- **Date et heure de la requête** : [Date: Mon, 9 March 2020 09:02:22 GMT](#)

Résumé simple : les headers **précisent le contexte et les conditions** de la requête pour que le serveur sache comment traiter la demande.

3. Corps de la requête :

- Présent uniquement pour certaines méthodes HTTP comme **POST** et **PUT**.
- Contient les **données envoyées au serveur**.
- Les données peuvent être au format **JSON**, **XML** ou autres, selon l'application.

Méthodes HTTP (types de requêtes) :

- Les requêtes HTTP utilisent des **méthodes** pour indiquer l'action à effectuer sur une ressource.
- Méthodes les plus courantes :
 - **GET** : récupérer des données
 - **POST** : créer de nouvelles données
 - **PUT** : mise à jour **complète** d'une ressource
 - **PATCH** : mise à jour **partielle** d'une ressource
 - **DELETE** : supprimer une ressource

Lien avec les opérations CRUD :

- **Create** → POST
- **Read** → GET
- **Update** → PUT ou PATCH
- **Delete** → DELETE

Le contenu d'une réponse HTTP

Réponse HTTP :

- Fournit des informations sur le **résultat de la requête** envoyée par le client.

Élément clé :

- **Ligne de statut HTTP** :
 - Première ligne de la réponse.
 - Contient le **code de statut** et une **description**.
 - Exemples :
 - **HTTP/1.1 200 OK** → requête réussie
 - **HTTP/1.1 404 Not Found** → ressource introuvable

Code de statut HTTP :

- Nombres à **trois chiffres** indiquant le résultat d'une requête.
- Aident à comprendre la réponse du serveur.

Catégories principales :

- **100–199** : réponses informatives
- **200–299** : requêtes réussies
 - **200 OK** : requête réussie avec données
 - **201 Created** : ressource créée
 - **204 No Content** : succès sans contenu
- **300–399** : redirections
- **400–499** : erreurs du client
 - **400 Bad Request** : requête invalide
 - **401 Unauthorized** : authentification manquante
 - **403 Forbidden** : accès refusé
 - **404 Not Found** : ressource introuvable
- **500–599** : erreurs du serveur
 - **500 Internal Server Error** : erreur interne du serveur

En-têtes de réponse :

- Fournissent des **informations supplémentaires** sur la réponse du serveur et sa gestion.
- Peuvent indiquer : le serveur utilisé, le type de contenu, les cookies, la date, la sécurité, etc.

Exemples courants :

- **Content-Type** : type du contenu renvoyé (`application/json`, `text/html`)
- **Server** : serveur web utilisé
- **Date** : date et heure de génération de la réponse
- **Set-Cookie** : définit des cookies stockés côté client

Corps de la réponse :

- Contient les **données renvoyées par le serveur**.
- Peut être de différents types : **HTML**, **JSON**, **XML**, **texte brut**, ou **données binaires** (images, fichiers).
- Son contenu dépend de la **requête envoyée** et de la **logique du serveur**.

MAGUEMOUN SAMY

Chapitre 2 : Programmation Asynchrone et Requêtes HTTP en JavaScript

partie 1 : la programmation asynchrone

Programmation synchrone

- Les tâches s'exécutent **dans un ordre précis**.
- Une tâche doit être **terminée avant que la suivante ne commence** (modèle bloquant).
- Si une opération est lente (lecture de fichier, requête réseau), **tout le programme attend**.
- Le flux d'exécution est **linéaire et prévisible**.
- **Plus simple à comprendre et à déboguer**, mais moins performante pour les opérations longues.

Programmation asynchrone

- Les tâches peuvent s'exécuter **indépendamment**, sans attendre la fin des autres.
- Modèle **non bloquant** : le programme continue pendant que des opérations longues (I/O, requêtes réseau) s'exécutent en arrière-plan.
- **Meilleures performances**, surtout pour les opérations lentes.
- **Meilleure réactivité** : l'interface n'est pas bloquée.
- **Optimisation des ressources** grâce à l'exécution simultanée de plusieurs tâches.
- **Plus complexe à comprendre et à déboguer**.
- Le flux d'exécution **n'est pas linéaire**.

Pourquoi l'Asynchronisme est important en JavaScript ?

- **JavaScript est monothread** : il n'exécute qu'une seule tâche à la fois.
- Sans asynchronisme, toute opération longue bloquerait l'exécution du programme.

Dans le navigateur

- Le thread principal gère l'interface utilisateur.
- Une opération bloquante figerait la page (clics, animations, affichage).
- L'asynchronisme permet de garder une interface fluide pendant les requêtes réseau ou autres tâches longues.

Côté serveur (Node.js)

- Node.js est aussi monothread.
- L'asynchronisme permet de traiter **des milliers de requêtes simultanément** sans bloquer le serveur.
- Il améliore les performances et la capacité à gérer de fortes charges.

En résumé

L'asynchronisme est essentiel pour éviter le blocage, garantir une bonne expérience utilisateur et assurer des applications JavaScript performantes et scalables.

Exemple de lecture de fichiers

- Module **fs** : permet de lire des fichiers.
- Deux méthodes principales :
 1. **readFile** → lecture **asynchrone** (non bloquante)
 2. **readFileSync** → lecture **synchrone** (bloquante)

blocking-example.js

Étape	Code	Action	Affichage sur le terminal
1	<code>console.log("Début");</code>	Affiche le début du programme	Début
2	<code>var data1 = fs.readFileSync("file1.txt");</code>	Lit synchrone le fichier <code>file1.txt</code> (bloquant)	–
3	<code>console.log("--Données du file 1 :");</code>	Indique que les données du fichier 1 vont suivre	--Données du file 1 :
4	<code>console.log(data1.toString());</code>	Affiche le contenu du fichier 1	aaaaaaaaaaaaaaaaaaaaa aaaaaaaaaaaaaaaaaaaaa aa ...
5	<code>console.log("Fin");</code>	Indique la fin de la lecture du fichier 1	Fin
6	<code>console.log("???");</code>	Affiche un séparateur avant le fichier 2	???
7	<code>var data2 = fs.readFileSync("file2.txt");</code>	Lit synchrone le fichier <code>file2.txt</code> (bloquant)	–
8	<code>console.log("--Données du file 2 :");</code>	Indique que les données du fichier 2 vont suivre	--Données du file 2 :
9	<code>console.log(data2.toString());</code>	Affiche le contenu du fichier 2	bbbbbbbbbbbbbbbbbbbbb bbbbbbbbbbbbbbbbbbbbb bb ...
10	<code>console.log("Fin");</code>	Indique la fin de la lecture du fichier 2	Fin

Non Blocking

- **Non bloquant** : le programme continue **immédiatement** l'exécution des lignes suivantes, **sans attendre** la fin de la lecture du fichier.
- **Callback** : une fonction qui sera exécutée **une fois la tâche terminée**.
- Permet de **meilleures performances** et une **interface réactive**.

Exemple de syntaxe :

Élément	Rôle
<code>fs.readFile('file1.txt', callback)</code>	Fonction asynchrone qui lit le fichier
<code>callback</code>	Fonction appelée après la fin de la lecture
Code après <code>fs.readFile</code>	S'exécute immédiatement , sans attendre la lecture

non-blocking-example.js

Étape	Code	Action	Affichage sur le terminal
1	<code>console.log("\n");</code>	Ligne vide pour démarquer le programme	(vide)
2	<code>fs.readFile("file1.txt", readFinishedFile1);</code>	Lit le fichier 1 de manière asynchrone (non bloquant)	—
3	<code>console.log("Lire file 2");</code>	Indique qu'on va lire le fichier 2	Lire file 2
4	<code>fs.readFile("file2.txt", readFinishedFile2);</code>	Lit le fichier 2 de manière asynchrone (non bloquant)	—
5	<code>console.log("-----Fin-----\n");</code>	Fin immédiate du programme principal	-----Fin-----

6	<code>readFinishedFile1(err, data)</code>	Callback exécutée quand la lecture du fichier 1 est terminée	- Données du file 1: AAAAAAAAAAAAAAAAAAAAA AAAAAAAAAAAAAAAAAAAAA ...
7	<code>readFinishedFile2(err, data)</code>	Callback exécutée quand la lecture du fichier 2 est terminée	- Données du file 2: BBBBBBBBBBBBBBBBBBBBB BBBBBBBBBBBBBBBBBBBBB ...

Points clés à retenir :

- Les appels à `fs.readFile` sont **non bloquants** → le programme continue immédiatement.
- Le **contenu des fichiers n'apparaît qu'après** l'affichage de "`-----Fin-----`".
- Les **callbacks** permettent de traiter les données **une fois la lecture terminée**.
- Montre le **principe fondamental de l'asynchronisme en Node.js**

Les mécanismes d'asynchronisme en javascript

La programmation asynchrone en JavaScript peut être réalisée principalement avec :

1. Callbacks

- Fonction passée en **argument** à une autre fonction asynchrone.
- Exécutée **une fois l'opération terminée**.
- Exemple : `fs.readFile('file.txt', callbackFunction)`.

2. Promises

Objet représentant un **résultat futur** ou un **échec** d'une opération asynchrone.

- Utilise `.then()` pour gérer le résultat et `.catch()` pour gérer les erreurs.
- Exemple :

```

async function fetchData() { try { const response = await fetch(url); const data = await
response.json(); console.log(data); } catch (err) { console.error(err); } }
fetch(url)
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));

```

3. async/await

- Syntaxe moderne et **lisible** pour travailler avec les Promises.
- Permet d'écrire du code asynchrone qui **ressemble à du synchrone**.

- Exemple :

```
async function fetchData() { try { const response = await fetch(url); const data = await response.json(); console.log(data); } catch (err) { console.error(err); } }
```

1. Le Callback : Le Premier Pilier de l'Asynchronisme

- Un **callback** est une fonction passée **en paramètre** à une autre fonction.
- Elle est **appelée** une fois que l'opération principale est **terminée**.
- Très utilisé en JavaScript et dans les environnements asynchrones.
- Permet de gérer les **opérations longues** :
 - requêtes réseau
 - lecture de fichiers
 - interactions avec des bases de données

En bref : le callback permet de continuer l'exécution du programme **sans bloquer** pendant que l'opération asynchrone s'exécute.

Exemple 01: `addEventListener`

- **Objectif** : afficher un titre caché lorsqu'on clique sur un bouton.

```
<h1 id="caché" style="visibility: hidden;">Je suis le titre qui était caché</h1>
<button id="btn">Clique Me</button>
<script>
  // Récupération de l'élément <h1>
  var h1 = document.getElementById("caché");
  // Définition du callback
  let callback = function () {
    h1.style.visibility = 'visible'; // rend le titre visible
  };
  // Attachement du callback à l'événement click du bouton
  document.getElementById("btn").addEventListener("click", callback);
</script>
```

Explications :

- `callback` est la fonction qui sera exécutée quand l'utilisateur clique sur le bouton.
- `addEventListener("click", callback)` enregistre la fonction pour l'événement click.
- Tant que l'utilisateur n'a pas cliqué, le titre reste **caché** (`visibility: hidden`).

Principe clé : le callback permet d'exécuter du code **en réponse à un événement**, ce qui est un exemple d'**asynchronisme côté navigateur**.

Rappel : La fonction `setTimeout()`

- **Fonction asynchrone** : exécute un **callback** après un délai spécifié (en ms).
- Le **fil principal** continue de tourner pendant le délai.
- Utile pour simuler des opérations longues ou différées.

Syntaxe :

`setTimeout(fonction, delay);`

Exemple :

Code	Affichage terminal
<code>console.log("Début");</code>	Début
<code>setTimeout(function() { console.log("Exécution après 1 seconde"); }, 1000);</code>	(rien immédiatement)
<code>console.log("Fin");</code>	Fin
(callback de <code>setTimeout</code> exécuté après 1s)	Exécution après 1 seconde

Exemple 03:

Objectif

Afficher **4 titres cachés** un par un, toutes les secondes.

HTML

```
<h1 id="titre1" style="visibility: hidden;">Titre 01</h1>
<h1 id="titre2" style="visibility: hidden;">Titre 02</h1>
<h1 id="titre3" style="visibility: hidden;">Titre 03</h1>
<h1 id="titre4" style="visibility: hidden;">Titre 04</h1>
```

Code JavaScript avec callbacks

```
setTimeout(() => {
  document.getElementById('titre1').style.visibility = 'visible';
  setTimeout(() => {
    document.getElementById('titre2').style.visibility = 'visible';
    setTimeout(() => {
      document.getElementById('titre3').style.visibility = 'visible';
      setTimeout(() => {
        document.getElementById('titre4').style.visibility = 'visible';
      }, 1000);
    }, 1000);
  }, 1000);
}, 1000);
```

Explications

- Chaque `setTimeout` attend que le précédent se termine pour afficher le titre suivant.
- Le délai est défini par le paramètre `1000` (1 seconde).

- Ce type de code est **difficile à lire et à maintenir**, surtout si on veut changer un délai ou ajouter plus d'étapes.

Problème clé

- L'imbrication des callbacks crée ce qu'on appelle « **l'enfer des callbacks** » : code lourd, peu lisible et fragile.

2. Les promesses (promise) : Simplifier la Gestion Asynchrone

- Les **Promises** sont des objets JavaScript qui rendent le code asynchrone **plus lisible et puissant** que les callbacks.
- Elles permettent de **ne pas bloquer l'interpréteur JavaScript** lors de l'exécution de tâches asynchrones.
- Lorsqu'une opération asynchrone est lancée, elle **retourne immédiatement une Promise** : un objet qui promet qu'un résultat sera disponible plus tard.
- Une Promise peut être :
 - **resolve** : opération réussie → renvoie un résultat.
 - **reject** : opération échouée → renvoie une erreur.
- Pour utiliser une Promise :
 - **.then()** → code exécuté lorsque la Promise est résolue avec succès.
 - **.catch()** → code exécuté lorsque la Promise est rejetée (erreur).

Syntaxe d'une Promise

```
new Promise(function (resolve, reject) {
  // Code asynchrone ici
  // Ne sera pas exécuté immédiatement, mais mis dans la file d'attente asynchrone

  if (isSuccess) {
    resolve(); // Promise résolue avec succès
  } else {
    reject(); // Promise rejetée en cas d'erreur
  }
});
```

Explications

- **new Promise(...)** → crée un **objet Promise**, représentant une opération asynchrone.
- La fonction passée en argument prend **2 paramètres** :
 - **resolve** → appelé si l'opération réussit.
 - **reject** → appelé si l'opération échoue.
- Le code **à l'intérieur** n'est pas exécuté immédiatement, il est mis dans la **file des opérations asynchrones**.

Exemple de Promise avec gestion succès / échec

```
// Création d'une Promise
let p = new Promise(function (resolve, reject) {
```



```

// Code asynchrone simulé
let isSuccess = true;
if (isSuccess) {
    resolve(); // opération réussie
} else {
    reject(); // opération échouée
}
});
// Gestion du résultat de la Promise
p
.then(() => {
    // Exécuté si la Promise est résolue
    console.log("calling then");
})
.catch(() => {
    // Exécuté si la Promise est rejetée
    console.log("error");
});

```

Explications

- `new Promise(...)` → crée l'objet Promise.
- `resolve()` → déclenche le `.then()` une fois l'opération terminée avec succès.
- `reject()` → déclenche le `.catch()` si l'opération échoue.
- Le `.then()` et `.catch()` permettent de **définir clairement le comportement après l'asynchrone**.

Exemple de Promise avec résultat

```

let p = new Promise(function (resolve, reject) {
    // Code asynchrone simulé
    let isSuccess = true;
    if (isSuccess) {
        resolve("Je suis le texte de resolve"); // valeur transmise à .then()
    } else {
        reject("Je suis le texte de reject"); // valeur transmise à .catch()
    }
});
p
.then((resolveMsg) => {
    // Exécuté si la Promise est résolue
    console.log(resolveMsg); // Affiche : "Je suis le texte de resolve"
})
.catch((errorMsg) => {
    // Exécuté si la Promise est rejetée
    console.log(errorMsg); // Affiche : "Je suis le texte de reject"
});

```

Explications

- `resolve(valeur)` → envoie une **valeur** à `.then()`.
- `reject(valeur)` → envoie une **valeur** à `.catch()`.
- Cela permet de **transmettre des données ou messages** après l'opération asynchrone.

En résumé : les Promises peuvent non seulement indiquer le succès ou l'échec, mais aussi renvoyer un **résultat ou un message** exploitable dans `.then()` ou `.catch()`.

Exemple d'asynchronisme avec Promise

```
let p = new Promise(function (resolve, reject) {
  console.log("Lancement de la requête en asynchrone");

  // Code asynchrone simulé : 2 secondes
  setTimeout(() => {
    console.log("Données prêtes...");
    resolve(); // déclenche le .then() après 2s
  }, 2000);
});
console.log("Le pg se poursuit...");
p
.then(() => {
  console.log("Affichage des données récupérées"); // exécuté après resolve()
})
.catch(() => {
  console.log("erreur"); // exécuté si reject()
});
console.log("Autres instructions du pg...");
```

Rendu attendu dans la console

```
Lancement de la requête en asynchrone
Le pg se poursuit...
Autres instructions du pg...
Données prêtes...
Affichage des données récupérées
```

Explications

- Le code **synchronisé** continue à s'exécuter **pendant l'attente** de la Promise (`console.log("Le pg se poursuit...")` et `"Autres instructions..."`).
- La fonction `setTimeout` est **asynchrone**, donc son code (`resolve()` et `"Données prêtes..."`) s'exécute **après 2 secondes**.
- `.then()` ne s'exécute qu'**après la résolution de la Promise**, illustrant clairement le **flux non bloquant**.

Exemple : Promise pour afficher un titre après un délai

```

let p = new Promise(function (resolve, reject) {
  // Code asynchrone simulé
  setTimeout(() => {
    // Message affiché dans la console après 3 secondes
    console.log("The title 1 is in the screen");
    // resolve() n'est pas appelé → .then() ne s'exécutera pas
    // resolve();
  }, 3000);
});
p
.then(() => {
  // Ce code s'exécute uniquement si resolve() est appelé
  document.getElementById('titre1').style.visibility = 'visible';
})
.catch(() => {
  console.log("erreur"); // exécuté si reject() est appelé
});

```

Explication clé

- **setTimeout** affiche "The title 1 is in the screen" après 3 secondes.
- Le **titre ne devient pas visible** car **resolve()** n'a pas été appelé.
- **.then()** ne s'exécute **que si la Promise est résolue**.
- Si tu veux que le titre apparaisse après 3 secondes, il faut **décommenter resolve()** dans le **setTimeout**.

Que se passera-t-il si on tape ce code? A quel moment sera affiché le message : "The title 02 is in the screen " ? Pourquoi?

```

let p = new Promise(function (resolve, reject) {
  setTimeout(() => {
    document.getElementById('titre1').style.visibility = 'visible';
    resolve();
  }, 2000);
});
p
.then(() => {
  setTimeout(() => {
    document.getElementById('titre2').style.visibility = 'visible';
  }, 2000);
})
.then(() => {
  setTimeout(() => {
    document.getElementById('titre3').style.visibility = 'visible';
  }, 2000);
})
.catch(() => {
  console.log("erreur");
});

```

Ce qui se passe

1. **Création de la Promise p**
 - `setTimeout` déclenche l'affichage du **titre 1** après **2 secondes**.
 - Après avoir affiché le titre 1, la Promise est **résolue** (`resolve()`).
2. **Premier `.then()`**
 - Dès que la Promise est résolue, le premier `.then()` s'exécute.
 - À l'intérieur, il y a un **`setTimeout` de 2 secondes** pour afficher le **titre 2**.
 - Important : le `.then()` se termine immédiatement après avoir défini le `setTimeout`.
→ La Promise renvoyée par ce `.then()` est **résolue immédiatement**, car aucun `resolve` n'est utilisé à l'intérieur.
3. **Deuxième `.then()`**
 - Comme la Promise du premier `.then()` est déjà résolue, le deuxième `.then()` s'exécute **immédiatement** après.
 - Il définit son propre `setTimeout` de 2 secondes pour afficher le **titre 3**.

Conséquence sur l'affichage

- **titre1** → après **2 secondes**.
- **titre2** → **2 secondes après le début du premier `.then()`**, donc **environ 4 secondes après le lancement du script**.
- **titre3** → **2 secondes après le lancement du deuxième `.then()`**, mais celui-ci se déclenche immédiatement après le premier `.then()`.
→ Donc **titre2 et titre3 apparaissent presque en même temps, 2 secondes après le titre1**, car les `setTimeout` sont démarrés **simultanément**, pas en chaîne.

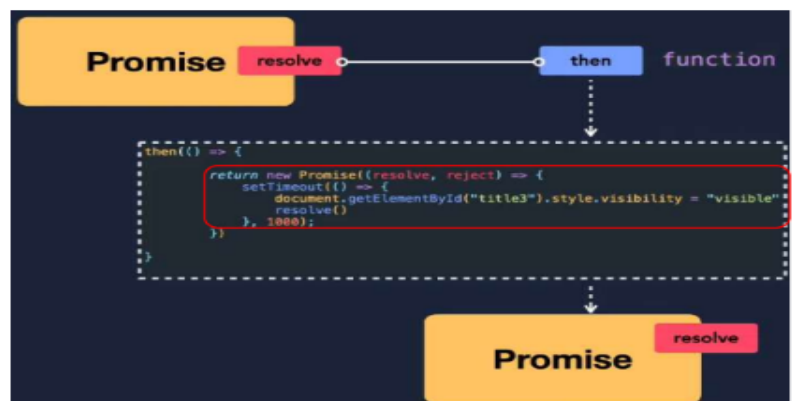
Réponse à la question

- Le message "The title 02 is in the screen" apparaîtra **environ 4 secondes après le lancement du script**, en même temps que le titre 3.
- **Pourquoi ?**
Parce que chaque `.then()` démarre un `setTimeout` mais ne renvoie pas de **Promise**, donc le deuxième `.then()` ne "s'attend pas" à la fin du `setTimeout` du premier.

Solution : l'affichage du titre 3 doit être une promesse dans la méthode `then` de la promesse précédente.

Exemple 1 : Chaînage simple de Promises

```
<script>
new Promise(function (resolve, reject) {
```




```

setTimeout(() => {
  document.getElementById('titre1').style.visibility = 'visible';
  resolve();
}, 1000);
}).then(() => {
  new Promise(function (resolve, reject) {
    setTimeout(() => {
      document.getElementById('titre2').style.visibility = 'visible';
      resolve();
    }, 1000);
  }).then(() => {
    setTimeout(() => {
      document.getElementById('titre3').style.visibility = 'visible';
    }, 1000);
  });
});
</script>

```

Fonctionnement :

- **titre1** apparaît après 1 seconde.
- Ensuite, **titre2** apparaît après 1 seconde de plus.
- Enfin, **titre3** apparaît après encore 1 seconde.
- Chaque **.then()** attend que la Promise précédente soit résolue avant de s'exécuter.

Exemple 2 : Chaînage avec console.log pour suivi

```

<script>
new Promise(function (resolve, reject) {
  setTimeout(() => {
    document.getElementById('titre1').style.visibility = 'visible';
    resolve();
  }, 1000);
}).then(() => {
  console.log("The title 1 is in the screen");
  return new Promise(function (resolve) {
    setTimeout(() => {
      document.getElementById('titre2').style.visibility = 'visible';
      resolve();
    }, 1000);
  });
}).then(() => {
  console.log("The title 2 is in the screen");
  return new Promise(function (resolve) {
    setTimeout(() => {
      document.getElementById('titre3').style.visibility = 'visible';
      resolve();
    }, 1000);
  });
});

```

dans l'ordre après 1 seconde chaque. Les logs de la console montrent clairement **quand** le code est exécuté.

Le code est plus lisible que des callbacks imbriqués. Il est facile pour changer le délai d'un titre sans avoir à modifier le code.

On peut facilement attendre de gérer l'asynchronisme **de manière asynchrone** avec **après la résolution de la promesse**. On peut utiliser `setTimeout` avec `console.log` et ajuster les délais. C'est beaucoup plus maintenable que des callbacks imbriqués.

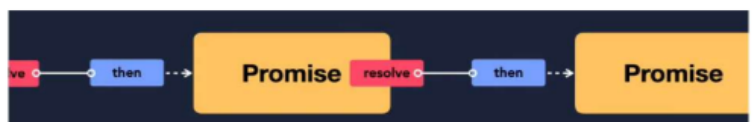
Fonctionnement :

- Chaque titre apparaît **dans l'ordre** après 1 seconde chacun.
- Les messages dans la console montrent clairement **quand chaque titre devient visible**.
- Le chaînage rend le code plus lisible que des callbacks imbriqués et **facile à modifier**, par exemple pour changer le délai d'un titre sans casser la suite.

Résumé général :

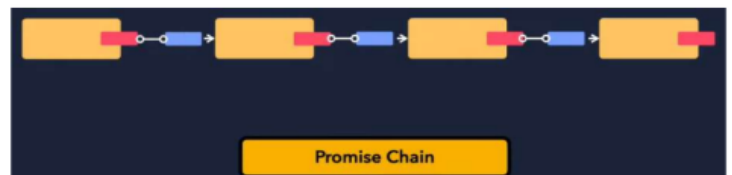
- Les Promises permettent de gérer l'asynchronisme **de manière ordonnée**.
- Chaque `.then()` s'exécute **après la résolution de la promesse précédente**.
- On peut suivre le flux avec `console.log` et ajuster les délais facilement.
- C'est plus lisible et maintenable que des callbacks imbriqués (callback hell).

Chaînage des promesses :



Problème :

Quand on imbrique plusieurs fonctions dans des `.then()` sans `return`, le code devient **compliqué et difficile à lire**, un peu comme les callbacks imbriqués (callback hell).



Solution :

Utiliser **return** dans les **.then()** pour renvoyer la **Promise suivante** dans la chaîne.

- Cela permet au **.then()** suivant de **s'exécuter seulement après la résolution** de la Promise renvoyée.
- On obtient un **chaînage linéaire et lisible**, facile à maintenir et à modifier.

Problème d'imbrication de plusieurs niveaux de fonctions dans le then :

But : éviter le problème d'imbrication (callback hell ou promesse hell) et rendre le code **lisible et maintenable**.

- Chaque `.then()` **retourne une nouvelle promesse** avec `return`.
- La promesse suivante ne s'exécute qu'après la **résolution** de la promesse précédente.
- Cela permet un **enchaînement linéaire**, chaque étape dépend de la précédente, mais reste lisible.

Exemple avec 4 titres :

```
let p = new Promise(function (resolve) {
  setTimeout(() => {
    document.getElementById('titre1').style.visibility = 'visible';
    resolve();
  }, 1000);
});
p
.then(() => {
  return new Promise(function (resolve) {
    setTimeout(() => {
      document.getElementById('titre2').style.visibility = 'visible';
      resolve();
    }, 1000);
  });
})
.then(() => {
  return new Promise(function (resolve) {
    setTimeout(() => {
      document.getElementById('titre3').style.visibility = 'visible';
      resolve();
    }, 1000);
  });
})
.then(() => {
  return new Promise(function (resolve) { // on retourne encore la promesse
    setTimeout(() => {
      document.getElementById('titre4').style.visibility = 'visible';
      resolve();
    }, 1000);
  });
});
```

Remarque :

- Le mot-clé `return` dans chaque `.then()` est **crucial** pour que la promesse suivante attende la fin de la précédente.

- Sans **return**, certaines étapes pourraient s'exécuter **en parallèle** ou **dans le mauvais ordre**.

3. **async et await dans javascript : Une Syntaxe Moderne pour l'Asynchronisme**

- **async** : marque une fonction comme asynchrone et lui permet d'utiliser **await**.
- **await** : bloque l'exécution de la fonction **asynchrone** jusqu'à ce que la promesse soit résolue, puis continue avec le résultat.

Exemple : affichage progressif de 4 titres

```
// Fonction qui retourne une promesse pour afficher un titre
function getTitre(num) {
  return new Promise(function (resolve, reject) {
    setTimeout(() => {
      document.getElementById('titre' + num).style.visibility = 'visible';
      resolve();
    }, 2000);
  });
}
// Fonction asynchrone pour afficher les titres l'un après l'autre
async function affTitre(num) {
  for (let i = 1; i <= num; i++) {
    await getTitre(i); // attend que le titre précédent soit affiché
  }
}
// Appel de la fonction
affTitre(4);
```

Explication :

1. **getTitre(num)** retourne une promesse qui s'exécute après 2 secondes et rend visible un titre.
2. **affTitre(num)** est **asynchrone** et utilise une boucle **for**.
3. **await getTitre(i)** attend la résolution de la promesse avant de passer au titre suivant.
4. Les 4 titres s'affichent **dans l'ordre**, chaque 2 secondes.

Avantages par rapport aux callbacks ou promesses en chaîne :

- Code **plus lisible** et linéaire.
- Plus facile à maintenir et modifier les délais ou l'ordre.
- Évite le "callback hell" ou le chaînage compliqué des **.then()**.

Comparaison entre les 3 méthodes :

Le code le plus **concis et lisible** est celui avec **async/await**.

Pourquoi :

Méthode	Lisibilité	Concision	Maintenance
Callbacks	Très imbriqué (callback hell), difficile à lire	Long et lourd à modifier	Modifier le délai ou l'ordre est compliqué
Promesses (then)	Chaînage plus clair que callbacks	Moins imbriqué, mais beaucoup de répétition (new Promise)	Relativement facile, mais répétitif
async/await	Code linéaire, très lisible	Très court : boucle + await	Très facile à modifier le délai ou le nombre de titres

Conclusion : **async/await** est le plus concis, le plus clair et le plus facile à maintenir.

Partie 2 : Programmation des requêtes HTTP :

Les requêtes HTTP en javascript?

Les requêtes HTTP en JavaScript permettent au client (navigateur ou application) de communiquer avec un serveur pour envoyer ou recevoir des données. Chaque requête utilise une **URL** et un **type** (GET, POST, PUT, DELETE...). Le serveur répond avec des données (souvent au format JSON) si la requête réussit, ou un message d'erreur si elle échoue.

Les techniques de la programmation des requetes HTTP en js :

Les principales techniques pour effectuer des requêtes HTTP en JavaScript côté client sont :

1. **XMLHttpRequest** : l'ancienne méthode standard pour envoyer et recevoir des requêtes HTTP.
2. **Fetch API** : une API moderne, plus simple et basée sur les Promises, pour gérer les requêtes HTTP.
3. **Axios** : une bibliothèque externe qui simplifie l'envoi de requêtes et la gestion des réponses, compatible avec Promises.

1. XMLHttpRequest (AJAX)

HttpRequest (XHR / AJAX) :

- Permet d'envoyer des requêtes HTTP et de récupérer des données depuis un serveur **sans recharger la page**.
- Utilisé pour créer des pages web **dynamiques et interactives** (approche AJAX).
- Bien que son nom contienne "XML", il peut récupérer **tout type de données** (JSON, texte, HTML, etc.).
- Utile pour mettre à jour partiellement une page web **sans perturber l'expérience utilisateur**.

Exemple1: Comment envoyer une requête GET en JavaScript à l'aide de XMLHttpRequest ?

```
<body>
```

```

<h1>C'est un exercice pour découvrir l'objet Request</h1>
<div id="content"></div>
<script>
let readPosts = function () {
  let req = new XMLHttpRequest(); // Création de l'objet XHR
  req.open('GET', 'https://jsonplaceholder.typicode.com/posts'); // Préparation de la requête
  GET
  req.responseType = "json"; // On attend une réponse au format JSON
  req.send(); // Envoi de la requête

  let divContenent = document.getElementById("content");

  req.onload = () => { // Callback exécuté lorsque la réponse est reçue
    let posts = req.response;
    for (post of posts){
      console.log(post); // Affiche chaque post dans la console
      divContenent.innerHTML += `<h2>${post.title}</h2>`; // Affiche le titre dans la page
    }
  }
}
readPosts(); // Appel de la fonction pour lancer la requête
</script>
</body>

```

Explications :

1. `XMLHttpRequest()` : crée un objet pour envoyer la requête.
2. `req.open('GET', URL)` : configure le type et l'URL de la requête.
3. `req.responseType = "json"` : indique que la réponse sera du JSON.
4. `req.send()` : envoie la requête au serveur.
5. `req.onload` : fonction callback qui s'exécute **quand la réponse est prête**.
6. `divContenent.innerHTML += ...` : ajoute dynamiquement les titres récupérés dans la page.

Résultat attendu :

- La page affiche tous les titres des posts récupérés depuis l'API JSONPlaceholder.
- Chaque post est également affiché dans la console pour debug.

Code :

```

function createNewPost() {
  let req = new XMLHttpRequest(); // Création de l'objet XHR
  req.open('POST', 'https://jsonplaceholder.typicode.com/posts'); // Préparation de la
  requête POST
  req.responseType = "json"; // On attend une réponse JSON

  // Définition des en-têtes HTTP
  req.setRequestHeader('Accept', 'application/json');

```

```

req.setRequestHeader('Content-Type', 'application/json');

// Corps de la requête (données envoyées au serveur)
let bodyParams = `{
  "title": "yyoo",
  "body": "bhhhf",
  "userId": 1
}`;

req.send(bodyParams); // Envoi de la requête avec les données

req.onload = () => { // Callback exécuté lorsque la réponse est reçue
  if (req.status >= 201 && req.status <= 299) { // Vérifie si la création a réussi
    let posts = req.response;
    console.log(req.status); // Affiche le code HTTP (ex : 201)
    console.log(posts); // Affiche la réponse du serveur (nouveau post)
  } else {
    console.log("erreur"); // En cas d'erreur
  }
}
}
createNewPost(); // Appel de la fonction pour créer le post

```

Explications :

1. `req.open('POST', URL)` : configure la requête comme POST vers l'API.
2. `req.setRequestHeader()` : définit le type de données envoyées et acceptées (JSON).
3. `req.send(bodyParams)` : envoie le corps de la requête contenant le nouveau post.
4. `req.onload` : callback exécuté à la réception de la réponse.
5. `req.status` : permet de vérifier si la requête a réussi (201 = créé).
6. `req.response` : contient les données renvoyées par le serveur (le nouveau post créé).

Résultat attendu :

- Console affiche le code HTTP 201.
- Console affiche l'objet JSON du nouveau post créé.
- Si la requête échoue, affiche "erreur".

Exercice d'application :

Écrire le code JavaScript à l'aide de XMLHttpRequest qui permet de :

1. Modifier un user donné de l'API JSONPlaceholder en envoyant une requête PUT ou PATCH ?
2. Supprimer un user donné de l'API JSONPlaceholder en envoyant une requête DELETE?
3. Filtrer les ressources (query params)

❶ Modifier un user avec PUT ou PATCH

On utilise **PUT** pour remplacer toutes les données d'un utilisateur ou **PATCH** pour modifier seulement certaines propriétés.

```
function updateUser(userId) {
  let req = new XMLHttpRequest();
  // Utiliser PATCH pour modification partielle
  req.open('PATCH', `https://jsonplaceholder.typicode.com/users/${userId}`);
  req.responseType = "json";
  req.setRequestHeader('Content-Type', 'application/json');
  let bodyParams = JSON.stringify({
    name: "Nouveau Nom",
    email: "nouveauemail@example.com"
  });
  req.send(bodyParams);
  req.onload = () => {
    if (req.status >= 200 && req.status <= 299) {
      console.log("Utilisateur modifié : ", req.response);
    } else {
      console.log("Erreur lors de la modification");
    }
  }
}

updateUser(1); // Modifier l'utilisateur avec l'id 1
```

Explications :

- **PATCH** permet de modifier uniquement certains champs (**name**, **email**).
- **req.response** contient la nouvelle version de l'utilisateur.

2 Supprimer un user avec DELETE

```
function deleteUser(userId) {
  let req = new XMLHttpRequest();
  req.open('DELETE', `https://jsonplaceholder.typicode.com/users/${userId}`);
  req.send();
  req.onload = () => {
    if (req.status >= 200 && req.status <= 299) {
      console.log(`Utilisateur ${userId} supprimé`);
    } else {
      console.log("Erreur lors de la suppression");
    }
  }
}

deleteUser(1); // Supprimer l'utilisateur avec l'id 1
```

Explications :

- **DELETE** n'a pas besoin de corps (**body**).
- Vérifie le code HTTP pour confirmer la suppression.

3 Filtrer des ressources avec query params


```
function filterUsersByCity(city) {
  let req = new XMLHttpRequest();
  // Ajout d'un paramètre de requête ?city=LeNomDeLaVille
  req.open('GET', `https://jsonplaceholder.typicode.com/users?address.city=${city}`);
  req.responseType = "json";
  req.send();

  req.onload = () => {
    if (req.status >= 200 && req.status <= 299) {
      console.log(`Utilisateurs dans la ville ${city} : `, req.response);
    } else {
      console.log("Erreur lors de la récupération");
    }
  }
}
}
```

```
filterUsersByCity("Gwenborough"); // Exemple : filtre les utilisateurs de la ville
"Gwenborough"
```

Explications :

- Les paramètres après `?` dans l'URL permettent de filtrer les ressources côté serveur.
- Ici on filtre les utilisateurs par ville.

2. À l'aide de l'API Fetch :

- **Fetch** permet d'envoyer des requêtes HTTP et de récupérer des ressources réseau de manière **asynchrone**.
- Contrairement à **XMLHttpRequest**, Fetch utilise **les promesses**, ce qui rend le code plus lisible et facile à chaîner.

Syntaxe générale

```
fetch(url, {
  method: 'GET' | 'POST' | 'PUT' | 'PATCH' | 'DELETE',
  headers: {
    'Content-Type': 'application/json', // type des données
    // autres en-têtes si nécessaire
  },
  body: JSON.stringify({ key1: 'value1', key2: 'value2' }) // pour POST, PUT, PATCH
})
.then(response => response.json()) // convertir la réponse en JSON
.then(data => console.log(data)) // utiliser les données
.catch(error => console.error(error)); // gérer les erreurs
```

- Pour **une requête simple GET** :

```
fetch('http://www.site.com/users')
```

```
.then(response => response.json())
.then(data => console.log(data));
```

En résumé : Fetch est une méthode moderne pour envoyer des requêtes HTTP en JavaScript, utilisant les promesses pour gérer facilement les réponses et les erreurs.

Comment envoyer une requête GET en JavaScript à l'aide de l'API Fetch **Requête GET avec Fetch**

```
fetch("https://jsonplaceholder.typicode.com/users")
  .then((response) => {
    // Vérifie si la réponse HTTP est OK (statut 200–299)
    if (!response.ok) {
      throw new Error('Erreur de réseau: ' + response.status);
    }
    // Convertit la réponse en JSON
    return response.json();
  })
  .then((json) => {
    // Utilise les données JSON récupérées
    console.log(json[0].name); // Affiche le nom du premier utilisateur
  })
  .catch((error) => {
    // Gérer les erreurs éventuelles
    console.error('Erreur lors de la récupération des données:', error);
  });
```

Explications :

1. `fetch(url)` : envoie la requête GET à l'URL spécifiée.
2. `response.ok` : vérifie que le serveur a répondu correctement.
3. `response.json()` : convertit la réponse en objet JSON.
4. `.then(json => ...)` : traite les données récupérées.
5. `.catch(error => ...)` : capture les erreurs réseau ou de conversion.

C'est la méthode **moderne et asynchrone** pour récupérer des données depuis un serveur.

Remarque :

1. **Fetch retourne toujours une promesse :**
 - `fetch(url)` démarre le chargement d'une ressource et retourne une promesse résolue avec un objet `Response` lorsque la réponse arrive.
2. **Les erreurs HTTP ne rejettent pas la promesse :**
 - Même si le serveur renvoie un code 404 ou 500, la promesse est considérée comme **résolue**.
 - Il faut **vérifier manuellement** `response.ok` ou `response.status` pour détecter les erreurs HTTP.
3. **La promesse fetch ne se rejette que pour les problèmes réseau :**
 - Par exemple : absence de connexion, permissions refusées, etc.

- Dans ce cas, `.catch()` est exécuté.
4. **AbortController** :
- Permet d'annuler une requête fetch si nécessaire.
 - Fonctionne dans la plupart des contextes (`window` ou `WorkerGlobalScope`).

En pratique :

```
fetch("https://jsonplaceholder.typicode.com/users")
  .then(response => {
    if (!response.ok) { // vérifie les erreurs HTTP
      throw new Error('Erreur HTTP: ' + response.status);
    }
    return response.json();
  })
  .then(data => console.log(data))
  .catch(error => console.error('Erreur réseau ou autre:', error));
```

Point clé : `fetch()` ne détecte pas les erreurs HTTP automatiquement, seulement les problèmes réseau.

Comment envoyer une requête POST en JavaScript à l'aide de l'API Fetch ?

1. **URL** : `"https://jsonplaceholder.typicode.com/posts"`
2. **Options de la requête** :
 - `method: "POST"` → indique que l'on envoie des données au serveur.
 - `body: JSON.stringify({...})` → les données envoyées, encodées en JSON.
 - `headers` → précise le type de contenu : `application/json`.
3. **Gestion de la réponse** :
 - `.then(response => response.json())` → convertit la réponse brute en JSON.
 - `.then(json => console.log(json.title))` → utilise les données renvoyées.

Code complet :

```
fetch("https://jsonplaceholder.typicode.com/posts", {
  method: "POST",
  body: JSON.stringify({
    title: "Hello World",
    body: "My POST request",
    userId: 900
  }),
  headers: { "Content-type": "application/json; charset=UTF-8" }
})
  .then(response => response.json()) // Convertit la réponse en JSON
  .then(json => console.log(json.title)); // Affiche le titre de la réponse
```

Point clé : Avec `fetch()`, on récupère les données en deux étapes :

1. transformer la réponse en JSON,
2. utiliser ce JSON dans un `.then()` suivant.

3. À l'aide d'Axios

1. Qu'est-ce qu'Axios ?

- Bibliothèque JavaScript tierce pour envoyer des requêtes HTTP.
- Basée sur les **promises**, donc elle facilite la gestion asynchrone.
- Simplifie les appels par rapport à `XMLHttpRequest` ou `fetch()`.

2. Installation / usage côté navigateur :

- Ajouter le CDN dans le `<head>` :
`<script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>`

Syntaxe d'une requête POST :

```
axios.post('https://api.example.com/data', {  
  key1: 'value1',  
  key2: 'value2'  
})  
  .then(response => {  
    // Gérer la réponse ici  
    console.log(response.data);  
  })  
  .catch(error => {  
    // Gérer les erreurs ici  
    console.error(error);  
  });
```

Avantages :

- Gestion simplifiée des promesses et des erreurs.
- Supporte GET, POST, PUT, DELETE... facilement.
- Compatible navigateur et Node.js.

Comment envoyer une requête GET en JavaScript à l'aide de AXIOS ?

```
function getUsers() {  
  axios.get('https://jsonplaceholder.typicode.com/users') // Requête GET  
    .then((response) => {  
      // Réponse reçue, accès aux données via response.data  
      console.log(response.data[0].name); // Affiche le nom du premier utilisateur  
    })  
    .catch((e) => {  
      // Gérer les erreurs réseau ou autres  
      alert(e);  
    });  
}  
getUsers(); // Appel de la fonction
```

Explication :

1. `axios.get(url)` envoie une requête GET à l'URL donnée.
2. `.then()` : exécuté si la requête réussit, la réponse est disponible dans `response`.

3. `.catch()` : exécuté en cas d'erreur (ex : problème réseau).
4. `response.data` contient les données renvoyées par le serveur (ici, un tableau d'utilisateurs).

Comment envoyer une requête PUT en JavaScript à l'aide de AXIOS ?

```
axios.put("https://jsonplaceholder.typicode.com/posts/10", {
  title: "PUT request with Axios",
  body: "PUT request",
  userId: 10,
})
.then(function (response) {
  console.log(response.data.title); // Affiche le titre mis à jour
})
.catch((error) => console.log(error)); // Gérer les erreurs
```

Explication :

1. `axios.put(url, data)` : envoie une requête PUT à l'URL pour **mettre à jour** complètement la ressource avec les nouvelles données `data`.
2. `.then()` : exécuté si la requête réussit, `response.data` contient la ressource mise à jour.
3. `.catch()` : exécuté en cas d'erreur (ex : problème réseau ou mauvaise URL).

Comparaison fetch et axios :

1. Gestion des erreurs

- **Fetch :**
 - La promesse n'est **pas rejetée** automatiquement en cas d'erreur HTTP (ex. 404 ou 500).
 - Il faut vérifier manuellement `response.ok` ou `response.status` pour détecter les erreurs.
- **Axios :**
 - La promesse est **rejetée automatiquement** pour toutes les erreurs HTTP (4xx, 5xx).
 - Plus simple pour gérer les erreurs avec `.catch()`.

2. Accès aux données

- **Fetch :**
 - La réponse renvoyée est un objet `Response`.
 - Il faut appeler `.json()`, `.text()` ou `.blob()` pour obtenir les données.
- **Axios :**
 - La réponse est directement encapsulée dans un objet JavaScript exploitable.
 - Les données sont disponibles via `response.data` sans transformation supplémentaire.

Résumé

- **Axios** : plus simple, gère automatiquement les erreurs HTTP et fournit directement les données.
- **Fetch** : natif, léger, mais nécessite plus de code pour vérifier les erreurs et extraire les données.

MAGUEMOUN SAMY

Chapitre 3: Techniques de programmation coté serveur avec nodejs

C'est quoi NodeJs ?

- **Type** : Environnement d'exécution (runtime) pour JavaScript côté serveur, **pas un framework**.
- **Fonction** : Permet d'exécuter du code JavaScript hors du navigateur.
- **Contenu** : Fournit un ensemble de **librairies JavaScript** pour développer des applications serveur.
- **Moteur JavaScript** : Utilise le moteur **V8 de Chrome** (très performant et open source) pour exécuter le code.
- **Création** : Développé par Ryan Dahl en 2009, en **C++**, pour créer des applications côté serveur.
- **Plateforme** : **Cross-platform** (Windows, macOS, Linux).

Environnement d'exécution

- **Définition** : C'est le lieu où un programme s'exécute pour produire son résultat (par exemple, une application).
- **Exemple classique** : Le navigateur web est un environnement d'exécution pour JavaScript.

Comment le navigateur interprète JavaScript ?

- Chaque navigateur possède son **propre moteur JavaScript** :
 - **V8** → Chrome, Node.js
 - **SpiderMonkey** → Firefox
 - **Chakra** → Edge (ancienne version)
- Autrefois, le même code JavaScript pouvait se comporter différemment selon le navigateur.
- Aujourd'hui, grâce aux **standards modernes et à la convergence des technologies**, le code est beaucoup plus cohérent sur tous les navigateurs.

Pourquoi Node JS ?

1. Utilisation de JavaScript côté serveur et client

- Permet d'utiliser le même langage pour le **front-end** et le **back-end** → réduit le coût de développement.
- Facilite la communication entre développeurs front-end et back-end.
- Même gestionnaire de packages (**npm**) pour le client et le serveur.

2. Très rapide

- Basé sur le moteur **V8** de Google (JIT → compilation juste-à-temps).
- Excellentes performances et vitesse d'exécution.
- Idéal pour les applications web **en temps réel** comme les chats.

3. Flexible

- Grâce à **npm**, on peut partager, mettre à jour et réutiliser des modules facilement.

4. Polyvalent

- Permet de développer des serveurs HTTP, API, applications de streaming, ou même des applications desktop.

5. Populaire et supporté

- Adopté par des géants comme **Microsoft, Google, Yahoo, Mozilla, GitHub**.
- Utilisé pour des applications connues comme **LinkedIn, Netflix, Uber, PayPal, Walmart**.

6. Communauté mondiale

- Grande disponibilité de **bibliothèques**, de modules et de ressources pour faciliter le développement.

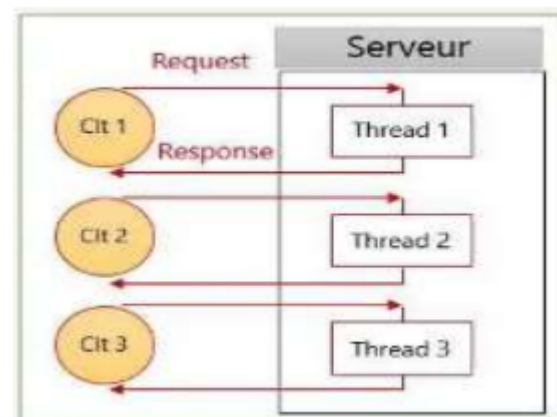
Modèle multi-threads bloquant (exemple : Apache/PHP)

1. Multithreading

- Chaque requête reçue par le serveur est assignée à un **thread**.
- Le serveur maintient un **pool de threads** (souvent des centaines).

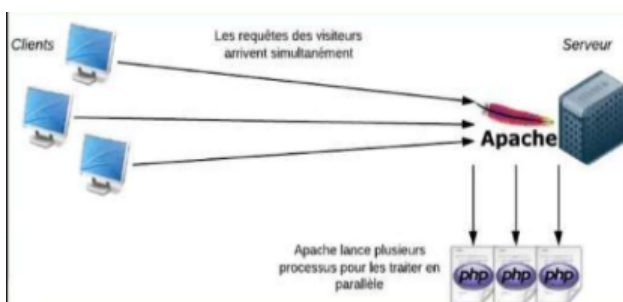
2. Exécution synchrone

- Le code PHP est exécuté **de manière bloquante** dans chaque thread.
- Tant qu'une requête n'est pas terminée, le thread reste occupé et **bloqué**.



3. Caractéristiques

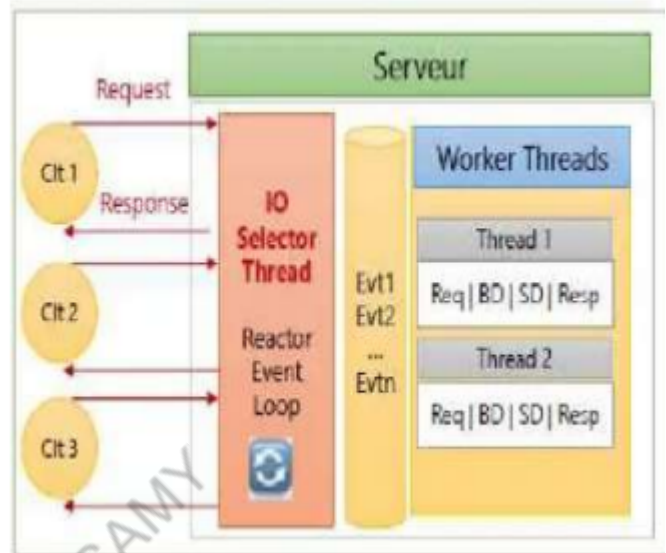
- Modèle classique et largement utilisé.
- Le blocage se produit **à l'intérieur du thread**, mais pas entre les threads.
- Limité par le nombre de threads disponibles → si tous les threads sont occupés, les nouvelles requêtes doivent attendre.



Node.js : Modèle Single

Modèle "single Thread non bloquant"

1. **Réception d'une requête**
 - La requête arrive et est reçue par le **thread principal**, appelé **Event Loop**.
2. **Tâches rapides**
 - Si la tâche est **rapide** (calcul simple), elle est exécutée **immédiatement** par l'Event Loop.
3. **Tâches I/O longues**
 - Si la tâche implique une **opération d'I/O** (lecture/écriture de fichiers, requête SQL, compression, etc.), elle est **déléguée à un Worker Thread** du **thread pool** géré par **libuv**.
4. **Non blocage**
 - Pendant qu'une tâche est en attente, le **thread principal continue à traiter d'autres requêtes**.
 - Le thread principal **n'est jamais bloqué**.
5. **Événements et callbacks**
 - Lorsque la tâche I/O est terminée, un **événement** est ajouté à la **file d'attente des événements (Event Queue)**.
6. **Exécution finale**
 - L'Event Loop récupère l'événement et exécute le **callback** ou la **promesse** associée pour renvoyer la réponse.



Opérations E/S (Entrée/Sortie)

- **Définition** : Communication entre le système et des périphériques externes (disques durs, réseau, clavier, imprimante...).
- **Caractéristiques** :
 - Plus lentes que les opérations en mémoire.
 - Peuvent impliquer des déplacements physiques ou des transmissions sur le réseau.
- **Optimisations possibles** :
 - Mise en cache.
 - Parallélisation des opérations.
 - Gestion **asynchrone** pour ne pas bloquer le système.

Avantages de Node.js

- Très léger : **un seul thread principal** suffit.

- Haute scalabilité : gère des **milliers de connexions simultanées**.
- Non bloquant : reste réactif même avec des E/S lentes.

Limites de Node.js

- Le code doit être **non bloquant**.
- Les calculs **CPU lourds** peuvent ralentir l'Event Loop.
- La programmation asynchrone est **plus complexe** : callbacks, promesses, async/await.

Installation de Node.js

1. Télécharger : <https://nodejs.org/fr/download/>
2. Vérifier l'installation : `node -v` # affiche la version de [Node.js](#)
3. Outils fournis après installation :

Node : exécute les scripts JS côté serveur. `node test.js`

- **npm (Node Package Manager)** :
 - Initialisation de projets.
 - Installation de dépendances.
 - Lancement de scripts.

Les modules :

- **Définition** :
Un module est une **unité de code encapsulée** (un fichier) pouvant contenir :
 - Variables
 - Constantes
 - Fonctions
 - Classes
- **Fonction** :
 - Permet de **réutiliser du code** dans différents fichiers.
 - Facilite l'organisation et la maintenance du code.

Types d'import/export

1. **CommonJS (Node.js traditionnel)**
 - Importer un module : `require('nom_du_module')`
 - Exporter des éléments : `exports.nom = valeur`
2. **ECMAScript Modules (ES6, navigateurs modernes et Node.js récents)**
 - Importer un module : `import nom from './module.js';`
 - Exporter un élément : `export const name = "NodeJS";`

Types de modules :

1/Modules du noyau (Core Modules)

- Modules inclus dans Node.js, **pas besoin d'installation**.

- On les utilise avec `require('nomModule')`.

Exemple : module **os**

Objectif : afficher l'architecture, le nombre de CPU, le hostname et le répertoire utilisateur.

Code source (exp1.js) :

```
const os = require("os");
console.log("Architecture:", os.arch());
console.log("Nombre CPU:", os.cpus().length);
console.log("Hostname:", os.hostname());
console.log("Répertoire:", os.homedir());
```

Exécution dans le terminal :

```
> node exp1.js
Architecture: x64
Nombre CPU: 4
Hostname: DESKTOP-30CGLO2
Répertoire: C:\Users\ULA
```

Résumé :

Les modules intégrés comme **os**, **fs**, **http**, etc., permettent d'accéder facilement aux fonctionnalités système et réseau **sans installer de dépendances supplémentaires**.

2/Modules gérés par NPM

- Ce sont des modules développés par la communauté, **non inclus dans Node.js**.
- NPM (Node Package Manager) permet de les **installer, gérer et mettre à jour** facilement.
- Gère automatiquement les **dépendances** : si un module a besoin d'un autre module, NPM le télécharge aussi.

Installation de modules

Installer localement (dans le projet)

```
npm install nomdumodule
```

ou équivalent court

```
npm i nomdumodule
```

Commandes utiles NPM

Commande	Description
npm install nomModule	Installer un module localement (dans <code>node_modules</code>)
npm install -g nomModule	Installer un module globalement
npm help nomCommande	Obtenir de l'aide pour une commande
npm ls	Lister les modules installés localement
npm ls -g	Lister les modules installés globalement
npm root	Localiser le dossier <code>node_modules</code>
npm init	Initialiser un projet Node.js (crée <code>package.json</code>)

3/Nos propres modules (Node.js)

Les **modules personnalisés** sont des fichiers JavaScript que **nous créons nous-mêmes** afin d'organiser et réutiliser le code.

Principe

- Un module est **un fichier .js**
- On **exporte** des éléments depuis ce fichier
- On les **importe** dans un autre fichier avec `require`
- On peut exporter : **fonctions, variables, constantes, objets, classes**

Exemple 3 : création d'un module

Fichier : `monModule.js`

```
var direBonjour = function () {
```

```
    console.log('Bonjour !');  
};  
  
var direByeBye = function () {  
    console.log('Bye bye !');  
};  
  
exports.direBonjour = direBonjour;  
exports.direByeBye = direByeBye;
```

Remarque importante :

`module.exports = ...` écrase l'export précédent.

Pour exporter **plusieurs éléments**, on utilise `exports.nom = ....`

Fichier : test.js

```
var monModule = require('./monModule');  
  
monModule.direBonjour();  
monModule.direByeBye();
```

NodeJS : permet de

Initialisation d'un projet Node.js

Node.js utilise **npm (Node Package Manager)** pour initialiser et gérer les projets.

Commandes d'initialisation

- `npm init`
→ Lance un **mode interactif** (questions/réponses).
- `npm init -y`
→ Initialise le projet **sans interaction** avec des valeurs par défaut.

Fichier package.json

La commande `npm init` génère le fichier **package.json**, qui est le **fichier de configuration principal** du projet.

Il contient notamment :

- **Nom du projet**
- **Version**
- **Description**
- **Auteur**
- **Licence**

- **Dépendances** du projet
- **Scripts npm** (ex : `start`, `test`, `build`, etc.)

Les scripts permettent d'automatiser des tâches comme :

- le démarrage du serveur, les tests, le déploiement.

Description du fichier `package.json`

Le fichier `package.json` est le fichier de configuration principal d'un projet Node.js.

Il permet principalement de :

- **Décrire le projet**
(nom, version, description, auteur, licence, etc.)
- **Lister les dépendances**
(modules nécessaires au fonctionnement du projet, installés via npm)

Il peut également contenir :

- les **scripts npm** (`start`, `test`, etc.), les informations de version, les dépendances de développement (`devDependencies`).

Une première application (un serveur web) avec Node.js

Avec **Node.js**, il n'est pas nécessaire d'utiliser un serveur web externe comme Apache.

Node.js permet de **créer directement son propre serveur HTTP**.

Principe général

- Les fonctionnalités de Node.js sont organisées en **modules**.
- Le module `http` permet de créer un serveur web.
- On crée un serveur avec `createServer()`.
- On lui fournit une **fonction callback** exécutée **à chaque requête client**.
- Le serveur est lancé avec `server.listen(port)`.

Exécution

Pour démarrer le serveur :

```
node nomFichier.js
```

Exemple de serveur HTTP simple

```
var http = require("http");

var server = http.createServer(function (req, res) {

  res.writeHead(200, { "Content-Type": "text/plain" });

  res.end("Salut tout le monde !");
```

```
});
```

```
server.listen(8080);
```

Explication

- `require("http")` : importe le module HTTP de Node.js.
- `http.createServer()` : crée le serveur.
- La fonction `(req, res)` est exécutée à chaque requête :
 - `res.writeHead(200, {...})` : définit le **code HTTP** et les **en-têtes**.
 - `res.end()` : envoie la réponse au client et termine la requête.
- `server.listen(8080)` : démarre le serveur sur le port **8080**.

Résultat

En accédant à <http://localhost:8080> dans le navigateur, le message suivant s'affiche :

Salut tout le monde !

Pourquoi la même réponse pour toutes les URLs ?

Dans la première version du serveur, **aucun test n'est fait sur l'URL demandée**.

Le serveur :

- reçoit une requête,
- exécute toujours le **même code**,
- renvoie donc **la même réponse**, quelle que soit l'URL :

```
/
/mepage
/dossier/autrepage
```

Pour Node.js, sans traitement spécifique, **toutes les requêtes sont gérées de la même façon**.

Solution : analyser l'URL demandée

Pour renvoyer une réponse différente selon la page, il faut :

1. **Récupérer le chemin de l'URL** tapée par l'utilisateur
2. **Tester ce chemin**
3. **Adapter la réponse**

Cela se fait grâce au module `url` et à la propriété `req.url`.

Récupération du chemin de l'URL

```
var page = url.parse(req.url).pathname;
```

Exemple : afficher l'URL demandée

```

var http = require('http');
var url = require('url');
var server = http.createServer(function (req, res) {
  var page = url.parse(req.url).pathname;
  console.log(page);
  res.writeHead(200, { "Content-Type": "text/plain" });
  res.write(
    'Bien le bonjour, vous avez tapé cette url : http://localhost:8080' + page
  );
  res.end();
});
server.listen(8080);

```

➡ Ici, la réponse dépend déjà de l'URL, car elle est **lue dynamiquement**.

Exemple : réponses différentes selon la page (routing simple)

```

var http = require('http');
var url = require('url');
var server = http.createServer(function (req, res) {
  var page = url.parse(req.url).pathname;
  console.log(page);
  res.writeHead(200, { "Content-Type": "text/plain" });
  if (page === '/') {
    res.write("Vous êtes à l'accueil, que puis-je pour vous ?");
  }
  else if (page === '/etudiant') {
    res.write("Vous êtes dans la page étudiant !");
  }
  else if (page === '/cours/web/') {
    res.write("Ici, vous trouverez les cours web !");
  }
  else {
    res.write("Page non trouvée");
  }

  res.end();
});

server.listen(8080);

```

Conclusion

- Node.js renvoie la même réponse par défaut car il **ne fait pas de routage automatiquement**
- Pour différencier les pages, il faut :
 - analyser `req.url`,
 - tester le chemin (`pathname`),
 - renvoyer une réponse adaptée

- Ce principe s'appelle le **routing** (routage), qui sera ensuite automatisé par des frameworks comme **Express.js**

Le framework Express.js : micro-Framework pour Node.js

Express.js est un **micro-framework** basé sur Node.js qui permet de développer des applications web et des API **plus rapidement et plus simplement** qu'avec Node.js "pur".

Pourquoi Express.js ?

- Fournit des **outils prêts à l'emploi** pour accélérer le développement
- Évite la programmation trop bas niveau de Node.js
- Simplifie :
 - la gestion des **routes (URLs)**
 - la navigation de l'application
 - l'utilisation des **moteurs de templates** (vues)
 - l'usage des **middlewares**

Mise en place d'un serveur express

Installation d'Express: `npm install express`

Création et démarrage d'un serveur Express

```
const express = require('express');
const app = express();
const port = 4000;

app.listen(port, () => {
  console.log(`Server is running on port ${port}`);
});
```

Gestion des routes simples :

Définition d'une route

Une **route** définit comment une application répond à une requête client selon :

- une **URL (chemin / URI)**
- une **méthode HTTP** (GET, POST, PUT, DELETE...)
- une **fonction de traitement** (handler ou middleware)

Éléments d'une route Express

1. **Path (chemin)** : l'URL partielle ciblée
2. **Méthode HTTP** : type de requête (GET, POST, etc.)
3. **Handler (callback)** : fonction exécutée pour traiter la requête et envoyer la réponse

Syntaxe :

app.method(path, handler)

- **method** : méthode HTTP (**get**, **post**, **put**, **delete**, ...)
- **path** : chemin de la ressource
- **handler** : fonction appelée lorsque la route est demandée

Exemple 1 : Route simple

```
const express = require('express');
const app = express();
const port = 4000;

app.get('/', function (req, res) {
  res.send('Vous êtes à la page d'accueil du serveur');
});

app.listen(port, () => {
  console.log(`Server is running on port ${port}`);
});
```

Lorsque l'utilisateur accède à **/**, le serveur renvoie un message texte.

Exemple 2 : Route API

```
app.get('/api/users', (req, res) => {
  const users = getUsersFromDatabase();
  res.json(users);
});
```

Cette route :

- répond aux requêtes **GET**
- retourne une **liste d'utilisateurs en JSON**
- est typiquement utilisée dans une API REST

Gestion des routes avec Express :

Types de routes

1. Routes statiques (Express.js)

Une **route statique** est une route dont le chemin (**URL**) est **fixe** et **ne contient aucune variable**.

Le serveur répond uniquement si l'URL demandée correspond **exactement** à la route définie.

Exemples de routes statiques

- **API REST** : **/api/version**, **/api/auth/login**
- **Application web** : **/**, **/home**, **/about**, **/contact**

Caractéristiques

- URL prédéfinie
- Pas de paramètres dynamiques
- Réponse identique à chaque appel
- Simple à comprendre et à maintenir

Exemple avec Express.js

```
const express = require('express');
const app = express();
// Route statique pour la page d'accueil
app.get('/', (req, res) => {
  res.send('Bienvenue sur la page d'accueil'); });
// Route statique pour la page "À propos"
app.get('/about', (req, res) => {
  res.send('Page À propos'); });
// Route statique pour la page "Contact"
app.get('/contact', (req, res) => {
  res.send('Page de contact'); });
// Démarrage du serveur
app.listen(3000, () => {
  console.log('Serveur en cours d'exécution sur http://localhost:3000'); });
```

2. Routes dynamiques (Express.js)

Définition

Une **route dynamique** est une route qui contient des **segments variables** dans son chemin. Ces segments permettent d'adapter la route à différentes valeurs et de cibler des ressources spécifiques (utilisateur, produit, etc.). Les valeurs dynamiques sont indiquées par des **paramètres de route**, précédés par :

Paramètres de route

- Les paramètres sont intégrés **dans le chemin de l'URL**.
- Ils sont automatiquement extraits par Express et accessibles via `req.params`.

Query parameters (paramètres de requête)

- Ajoutés **après le chemin**, précédés par `?`
- Format : `clé=valeur`
- Utilisés pour transmettre des informations supplémentaires (filtrage, tri, pagination...)
- Accessibles via `req.query`

Exemples de routes dynamiques

1. Route pour les utilisateurs

- Route : `/api/users/:id`
- URL correspondante : `/api/users/42`

2. Route pour les produits

- Route : `/api/products/:category/:id`
- URL correspondante : `/api/products/electronics/123`

Exemple avec Express.js

```
const express = require('express');
const app = express();
// Route dynamique avec paramètre
app.get('/api/users/:id', (req, res) => {
  const userId = req.params.id;
  res.send(`Utilisateur demandé : ${userId}`);
});
// Route dynamique avec plusieurs paramètres
app.get('/api/products/:category/:id', (req, res) => {
  const category = req.params.category;
  const id = req.params.id;
  res.send(`Produit ${id} dans la catégorie ${category}`);
});
app.listen(3000, () => {
  console.log('Serveur en cours d'exécution sur http://localhost:3000');
});
```

Résumé

- Les routes dynamiques utilisent des paramètres (`:param`)
- Elles permettent de gérer des ressources variables
- Express fournit `req.params` et `req.query` pour accéder aux valeurs
- Très utilisées dans les API REST

Passer des paramètres via Query Parameters avec Express.js

Définition

Un **query parameter** (paramètre de requête) permet d'ajouter des informations supplémentaires à une URL lors d'une requête HTTP.

Ils sont principalement utilisés pour :

- filtrer des résultats, trier des données, paginer des listes, préciser un comportement côté serveur.

Syntaxe

- Les paramètres sont ajoutés **après ?** dans l'URL
- Format : `clé=valeur`
- Plusieurs paramètres sont séparés par `&`

Exemples :

- `/api/users?id=42`
- `/api/products?category=electronics&price=low`
- `/blog?author=johndoe&year=2024`

Accès aux query parameters dans Express

Express met les paramètres de requête à disposition via l'objet **req.query**.

Exemple 1 : un seul paramètre

```
app.get('/users', (req, res) => {
  const userId = req.query.id;
  if (userId) {
    res.send(`Détails de l'utilisateur avec ID : ${userId}`);
  } else {
    res.send('Liste des utilisateurs');
  }
});
```

- URL `/users?id=42` → affiche l'utilisateur 42
- URL `/users` → affiche la liste des utilisateurs

Exemple 2 : plusieurs paramètres

```
app.get('/products', (req, res) => {
  const { category, price } = req.query;
  if (category && price) {
    res.send(`Produits de la catégorie "${category}" avec un prix "${price}"`);
  } else {
    res.send('Liste des produits');
  }
});
```

- URL `/products?category=electronics&price=low`
- Paramètres accessibles via `req.query.category` et `req.query.price`

Résumé

- Les query parameters servent à transmettre des données optionnelles
- Ils sont ajoutés après `?` dans l'URL
- Express les récupère avec `req.query`
- Très utilisés pour le filtrage, la recherche et la pagination dans les API REST

c. Paramètres dans le corps de la requête (POST, PUT, etc.):

Les **body parameters** permettent d'envoyer des informations plus complexes au serveur, comme des objets ou des ensembles de données.

Ils sont utilisés avec les requêtes HTTP **POST**, **PUT**, **PATCH**, etc.

Les données peuvent être structurées en **JSON**, **XML** ou formulaires encodés.

Pourquoi / utilité

- Permettent de transmettre des données structurées ou volumineuses au serveur.
- Nécessitent un middleware pour parser (convertir) le contenu en objets JavaScript exploitables.
- Idéal pour créer ou modifier des ressources côté serveur (ex : ajout d'utilisateur, login).

Exemple 1 : Formulaire HTML (POST)

Client :

```
<form action="http://127.0.0.1:3000/login" method="post">
  <label>Nom :</label>
  <input type="text" name="nom">
  <input type="submit" value="Envoyer">
</form>
```

Serveur :

```
app.post("/login", (req, res) => {
  console.log(req.body);
  res.json(req.body);
});
```

Sortie console :

```
Je viens de recevoir une requête
[Object: null prototype] { nom: 'Moussi' }
```

Le middleware transforme le JSON/formulaire en objet JavaScript utilisable.

Exemple 2 : Création d'un nouvel utilisateur

```
app.post('/users', (req, res) => {
  let data = { name: 'Samy', age: 22 };
  req.body = data;

  const { name, age } = req.body;
  res.send(`Nouvel utilisateur ajouté : ${name}, age ${age}`);
});
```

Points clés

- Les **body parameters** sont pour les données complexes envoyées dans POST/PUT.
- Express nécessite `express.json()` et `express.urlencoded()` pour parser le body.
- Accès via `req.body`.
- Très utilisés pour les formulaires, API REST, création ou modification de ressources.

Headers d'une requête HTTP :

Les headers (en-têtes) sont des informations envoyées par le client dans une requête HTTP, par exemple : type de navigateur, langue, type de contenu, etc.

Récupération des headers dans Express

- Méthode `req.get()` : récupère la valeur d'un header spécifique.
`const userAgent = req.get("user-agent");`
- Propriété `req.headers` : contient tous les headers sous forme d'objet.
`console.log(req.headers);`

Résumé :

il existe plusieurs méthodes pour récupérer les paramètres d'une requête HTTP :

a. Query parameters (URL)

- Ajoutés après `?` dans l'URL, format `clé=valeur`.
- Accès via `req.query`.

// URL : `http://localhost:3000/?prenom=Samy&nom=Ouari`

```
app.get('/', (req, res) => {  
  res.send(req.query.prenom); // "Samy"  
});
```

b. Route parameters (chemin de l'URL)

- Variables dans le chemin, précédées par `:`.
- Accès via `req.params`.

// URL : `http://localhost:3000/Samy/Ouari`

```
app.get('/:prenom/:nom', (req, res) => {  
  const prenom = req.params.prenom;  
  res.send('Salut ' + prenom + ' !'); // "Salut Samy !"  
});
```

c. Body parameters (POST, PUT)

- Données complexes envoyées dans le corps de la requête (JSON, formulaires).
- Middleware nécessaire : `express.json()` et `express.urlencoded()`.

```
app.post('/login', (req, res) => {  
  console.log(req.body);
```

```
res.json(req.body);  
});
```

d. Headers HTTP

- Informations sur la requête (ex. User-Agent, host, language).
- Accès via `req.get('nom-header')` ou `req.headers`.

```
const userAgent = req.get('user-agent');  
console.log(req.headers);
```

Méthodes de l'objet réponse (res)

Méthode	Description
<code>res.send()</code>	Envoie du texte, HTML ou JSON (détecte automatiquement le type).
<code>res.json()</code>	Envoie des données JSON (utile pour API REST).
<code>res.status()</code>	Définit le code HTTP de la réponse.
<code>res.redirect()</code>	Redirige vers une autre URL.
<code>res.render()</code>	Rend une vue/template via un moteur comme EJS.
<code>res.sendFile()</code>	Envoie un fichier au client.
<code>res.set()</code>	Définit un ou plusieurs en-têtes HTTP.

Exemples :

```
res.send('hello world');  
res.status(404).end();  
res.status(404).send('product not found.');
```



```
res.json({ name: "Samy" });  
  
res.redirect(301, 'http://example.com');
```

Types de données servies par un backend

- **Texte ou HTML** : simple texte ou contenu HTML (moins maintenable).
- **JSON** : format standard pour les données structurées.
- **API RESTful** : backend expose des ressources en JSON pour interagir avec les front-end ou applications mobiles.

Rendu côté serveur:

1. Servir des pages HTML statiques

- Utilise `res.sendFile()` pour envoyer des fichiers HTML au client.
- `__dirname` : variable globale Node.js contenant le chemin absolu du répertoire courant.

```
app.get('/accueil', (req, res) => {  
    res.sendFile(__dirname + '/views/accueil.html');  
    console.log(`Le répertoire courant est : ${__dirname}`);  
});
```

2. Servir des ressources statiques

- Middleware intégré `express.static()` pour servir fichiers CSS, JS, images, etc.
- Attachement via `app.use()`.
- Les fichiers doivent être placés dans un répertoire dédié, par exemple `public`.

```
const express = require('express');  
  
const app = express();  
  
app.use(express.static('public'));  
  
// URL : http://127.0.0.1:3000/bird.png  
  
console.log(`Le répertoire courant est : ${__dirname}`);  
  
app.listen(3000, () => console.log('Serveur à l'écoute'));
```

3. Servir des ressources dynamiques (Server Side Rendering)

- Génération de pages HTML **dynamiques** côté serveur.
- Avantages :
 - Allège la charge côté client

- Améliore le SEO (pages complètes dès la requête)
- Approche : utiliser un **squelette HTML** (template) dans lequel on insère des données dynamiques.
- Langages de templates : permettent d'intégrer facilement des données dans des fichiers HTML.

Rendu côté serveur (SSR) :

Le **Server Side Rendering (SSR)** consiste à générer des pages HTML dynamiques côté serveur avant de les envoyer au client.

- Avantages : pages prêtes à afficher, meilleure performance pour clients peu puissants, SEO optimisé.
- Différence avec CSR (Client Side Rendering) : ReactJS et autres frameworks front génèrent le rendu côté client.

Moteurs de templates supportés par Express

- **Handlebars**
- **EJS (Embedded JavaScript)**
- **JSP** (Java spécifique)
- Fonctionnalités communes : variables, conditions, boucles, inclusion de fichiers, etc.

Principe de fonctionnement

1. Le fichier Node.js (**app.js**) prépare les données à afficher.
2. Les données sont transmises au **template**.
3. Le template génère un HTML complet en utilisant les variables, conditions et boucles.
4. Le serveur envoie la page HTML finale au client.

Moteur de template EJS :

1. Définition

EJS (Embedded JavaScript) est un moteur de template pour Node.js qui permet de générer des pages HTML dynamiques en insérant des données et du code JavaScript directement dans des fichiers **.ejs**.

2. Syntaxe principale

Balise	Utilité
<code><% ... %></code>	Code JS exécuté mais non affiché dans le HTML
<code><%= ... %></code>	Affiche la valeur d'une variable dans le HTML
<code><%- include('partial.ejs') %></code>	Inclut un fichier partiel dans le template

<code><% if (condition) { %> ... <% } %></code>	Conditionnelle
<code><% for (let i=0; i<array.length; i++) { %> ... <% } %></code>	Boucle
<code><%# ... %></code>	Commentaire (non rendu)

3. Structure typique d'un projet EJS-Node.js

```

Projet/
├── views/
│   ├── partials/
│   └── pages/
├── public/
│   ├── images/
│   ├── css/
│   └── js/
├── node_modules/
├── app.js
└── package.json

```

4. Exemple simple

app.js :

```

app.get("/", (req, res) => {
  const data = {
    title: "Ma Page",
    greeting: "Bonjour",
    content: "Bienvenue sur ma page générée avec EJS."
  };
  res.render("template", data);
});

```

template.ejs :

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title><%= title %></title>
</head>
<body>
  <h1><%= greeting %></h1>
  <p><%= content %></p>
</body>
</html>

```

Les variables passées dans `res.render()` sont accessibles dans le template avec `<%= variable %>`.

5. Exemple avec route dynamique

```
app.get('/cours/:coursnum/lecon', (req, res) => {  
  res.render('lecon', { cours: req.params.coursnum });  
});
```

views/lecon.ejs :

```
<h1>Vous êtes dans une leçon</h1>  
<p>Vous êtes au cours n°<%= cours %></p>
```

Permet d'afficher dynamiquement le numéro du cours à partir de l'URL ou d'une base de données.

Résumé clé

- EJS permet d'insérer des **variables, conditions et boucles** dans du HTML.
- Routes Express peuvent passer des **données dynamiques** aux templates via `res.render()`.
- Supporte inclusion de fichiers partiels et rend possible le **Server Side Rendering** flexible.

Les middlewares

Un **middleware** est une fonction JavaScript capable d'interagir avec les **requêtes entrantes** (`req`) et les **réponses sortantes** (`res`) dans une application Express.

- Utilisé pour gérer des traitements communs à plusieurs routes (ex : logs, authentification, favicon, parsing des données).
- Permet de **factoriser le comportement** sur tous les points de terminaison de l'API.

Syntaxe :

```
const middleware = (req, res, next) => {  
  // Traitement quelconque sur req et res  
  next(); // Passe au middleware ou route suivant(e)  
};
```

Chargement du middleware

- Avec `app.use()` : exécuté **pour toutes les routes** après son ajout.

```
app.use(middleware);
```

- Avec route spécifique : exécuté **uniquement pour cette route**.

```
app.get('/', middleware, (req, res) => res.send('Page d'accueil'));
```

Exemple avec `app.use()`

```
const middleware = (req, res, next) => {  
  console.log("Middleware exécuté pour chaque requête");  
  next();  
};  
app.use(middleware);
```

```
app.get('/', (req, res) => res.send('Page d'accueil'));  
app.get('/about', (req, res) => res.send('À propos'));  
app.get('/contact', (req, res) => res.send('Contact'));  
app.get('/produit', (req, res) => res.send('Produit'));
```

Middleware exécuté **avant chaque route**.

Exemple sans **app.use()**, middleware spécifique par route

```
app.get('/', middleware, (req, res) => res.send('Page d'accueil'));  
app.get('/about', middleware, (req, res) => res.send('À propos'));  
app.get('/contact', middleware, (req, res) => res.send('Contact'));  
app.get('/produit', middleware, (req, res) => res.send('Produit'));
```

Middleware exécuté **uniquement sur les routes où il est déclaré**.

Questions fréquentes

- **Q1 : que se passe-t-il si **next()** est supprimé ?**
→ La chaîne de middleware ne continue pas, la route suivante ne s'exécute pas, le client reste bloqué.
- **Q2 : que se passe-t-il si **app.use(middleware)** est placé après **app.get()** ?**
→ Le middleware ne s'exécutera pas pour les routes définies avant, donc pas de traitement/log.

Cas d'utilisation de **app.use(middleware)**

1. Middleware global pour toutes les routes

- Déclaré avec **app.use(middleware)** sans route spécifique.
- S'exécute pour **toutes les requêtes** arrivant après cette instruction, quel que soit le chemin ou la méthode HTTP.

```
const middleware = (req, res, next) => {  
  console.log("Middleware exécuté pour chaque requête");  
  next();  
};  
app.use(middleware);  
app.get('/', (req, res) => res.send('Page d'accueil'));  
app.get('/about', (req, res) => res.send('À propos'));
```

Middleware exécuté **avant toutes les routes** / et /about.

2. Middleware pour une route spécifique (et ses sous-routes)

- Déclaré avec `app.use('/route', middleware)`.
- S'exécute uniquement pour la route spécifiée et ses sous-routes.

```
app.use('/about', (req, res, next) => {  
  console.log('Middleware exécuté pour /about uniquement');  
  next();  
});  
app.get('/about', (req, res) => res.send('À propos'));  
app.get('/', (req, res) => res.send('Page d'accueil'));
```

Middleware exécuté pour /about ou /about/sous-route, **pas pour /**.

3. Middleware pour une méthode HTTP spécifique

- Attacher le middleware **directement à une route avec une méthode** (GET, POST, etc.).

```
app.get('/about', middleware, (req, res) => {  
  res.send('Middleware exécuté avant cette route');  
});
```

Middleware exécuté **uniquement pour les requêtes GET vers /about**.

4. Récapitulatif

Déclaration	Portée / Usage
<code>app.use(middleware)</code>	Toutes les routes et méthodes HTTP
<code>app.use('/route', middleware)</code>	Routes spécifiques (et sous-routes)
<code>app.METHOD('/route', middleware, ...)</code>	

Les types de middlewares :

1. Middleware d'application :

Définition

- Un **middleware d'application** est exécuté **à chaque requête** reçue par l'API REST.
- Déclaré avec `app.use(middleware)`.
- Utile pour les traitements communs : logs, favicon, authentification, parsing, etc.

Exemple 1 : Middleware global (toutes les routes)

```
const express = require('express');  
const path = require('path');
```

```

const favicon = require('serve-favicon');
const app = express();
// Middleware pour servir l'icône favicon
app.use(favicon(path.join(__dirname, 'favicon.ico')));
// Middleware logger global
const logger = function(req, res, next) {
  const { method, url } = req;
  const date = new Date().toLocaleString();
  console.log(`${date} : ${method} ${url}`);
  next(); // Passe à la route suivante
};
app.use(logger);
// Route principale
app.get('/', (req, res) => {
  res.send('Hello, World!');
});

// Démarrage du serveur
const hostname = '127.0.0.1';
const port = process.env.PORT || 3000;
app.listen(port, hostname, () => {
  console.log(`Serveur démarré sur http://${hostname}:${port}`);
});

```

Exemple de sortie console :

```

Serveur démarré sur http://127.0.0.1:3000
03/12/2024 2:26:52 : GET /
03/12/2024 2:26:54 : GET /products?id=3&type=S

```

Remarque : si `next()` est supprimé, la route ne s'exécutera pas et la requête restera bloquée.

Exemple 2 : Middleware attaché à une route spécifique avec `app.use('/route', middleware)`

```

const express = require('express');
const app = express();
// Middleware exécuté uniquement pour /user et ses sous-routes
app.use('/user', function (req, res, next) {
  console.log('Hello World!');
  next(); // Passe ensuite au handler de la route
});
// Route /user
app.get('/user', function (req, res) {
  res.send('Il doit y avoir un Hello dans la console');
});
// Démarrage du serveur
app.listen(3000, () => {

```

```
console.log('Serveur démarré');
});
```

- Ici, le middleware **ne s'exécute que pour /user** (ou ses sous-routes).
- / ou d'autres routes **ne déclenchent pas le middleware**.

Exemple 3 : Middleware attaché directement à une route

```
const express = require('express');
const app = express();
// Middleware spécifique à une seule route
app.get('/user', function (req, res, next) {
  console.log('Hello World!');
  next(); // Passe ensuite au handler de la route
}, function (req, res) {
  res.send('Il doit y avoir un Hello World dans la console');
});

app.listen(3000, () => {
  console.log('Serveur démarré');
});
```

- Ici, le middleware est **exécuté uniquement pour cette route** et aucune autre.
- Permet de lier directement le middleware au traitement de la route.

Points clés

- `app.use(middleware)` → middleware global pour toutes les routes.
- `app.use('/route', middleware)` → middleware pour une route et ses sous-routes.
- `app.METHOD('/route', middleware, handler)` → middleware spécifique à une route avec méthode HTTP précise (GET, POST...).
- `next()` est indispensable pour passer au middleware ou au handler suivant.
- Permet de **factoriser les traitements communs** et d'améliorer la lisibilité du code.

2. Le middleware intégré :

- Middleware prédéfini par Express (depuis la version 4).
- Permet de **servir des fichiers statiques** : images, CSS, JS, HTML, etc.
- Déclaré avec `app.use(express.static('dossier'))`.

Exemple 1 : dossier public

```
app.use(express.static('public'));
```

- Si `public` contient `img/logo.png`, accessible via : `http://localhost:3000/img/logo.png`

Exemple 2 : dossier content


```
app.use(express.static("content"));
```

- Si `content/css/style.css` existe, accessible via `:http://localhost:3000/css/style.css`

Notes importantes

- On peut définir **plusieurs dossiers statiques** avec plusieurs `app.use(express.static(...))`.
- Express s'occupe de servir les fichiers directement, pas besoin de créer de route spécifique pour chaque fichier.

3. Le middleware tiers :

- Ce sont des **middlewares externes** fournis par des modules Node.js.
- Ils doivent être installés avec `npm` et apparaissent dans `node_modules`.
- Utilisés pour **ajouter des fonctionnalités spécifiques** sans réinventer la roue : logs, sécurité, authentification, parsing, etc.

Exemple : Morgan

- **Morgan** est un middleware pour **logger les requêtes HTTP**.
- Il enregistre : méthode HTTP, URL, code de statut, durée de réponse, etc.

Installation : `npm install morgan --save-dev`

Utilisation dans Express :

```
const express = require('express');
const morgan = require('morgan');
const app = express();
// Middleware Morgan pour journaliser les requêtes
app.use(morgan('dev'));
// 'dev' → format concis et coloré
// Autres formats : 'combined', 'tiny', etc.
// Route simple
app.get('/', (req, res) => {
  res.send('Hello, world!');
});
// Démarrage du serveur
app.listen(3000, () => {
  console.log('Serveur démarré sur http://localhost:3000');
});
```

Résultat :

- Chaque requête reçue est loggée dans la console automatiquement :

GET / 200 12ms

GET /style.css 304 2ms

- Les **middlewares tiers** permettent de **rajouter facilement des fonctionnalités avancées** sans coder tout soi-même.
- Morgan est un exemple classique, mais il en existe beaucoup : **helmet** (sécurité), **cors** (gestion des CORS), **body-parser** (parsing JSON/URL-encoded), etc.

4. Le middleware de traitement d'erreurs :

- Middleware conçu pour **gérer toutes les erreurs** de l'application.
- Intervient lorsqu'il y a :
 - une erreur dans une route,
 - une erreur dans un autre middleware,
 - ou une exception non prévue.
- **Spécificité** : les fonctions de traitement d'erreurs ont **4 arguments** (**err, req, res, next**) au lieu de 3.

Exemple simple

```
app.use((err, req, res, next) => {
  console.error(err.stack); // Affiche la stack trace
  res.status(500).send('Erreur interne du serveur'); // Envoie réponse 500
});
```

Exemple complet

```
const express = require('express');
const app = express();
// Route normale
app.get('/', (req, res) => {
  res.send('Page d'accueil');
});
// Route qui génère une erreur
app.get('/error', (req, res, next) => {
  const err = new Error('Quelque chose s'est mal passé');
  err.type = 'custom';
  next(err); // Passe l'erreur au middleware de traitement
});
// Middleware pour les routes non trouvées (404)
app.use((req, res) => {
  res.status(404).send('Ressource non trouvée !');
});
// Route asynchrone avec gestion des erreurs
/*
app.get('/async', async (req, res, next) => {
  try {
    await someAsyncOperation(); // Exemple : accès à la base de données
    res.send('Success');
  } catch (error) {
    next(error); // Passe l'erreur au middleware de traitement
  }
});
```

```

*/
// Middleware de gestion des erreurs
app.use((err, req, res, next) => {
  if (err.type == 'custom') {
    res.status(400).send('Erreur personnalisée : ' + err.message);
  } else {
    console.error(err.stack);
    res.status(500).send('Erreur interne du serveur');
  }
});
// Démarrage serveur
app.listen(3000, () => {
  console.log('Serveur démarré sur http://localhost:3000');
});

```

Points clés

- Middleware **avec 4 arguments** (`err`, `req`, `res`, `next`).
- Permet de gérer :
 - Erreurs personnalisées (`err.type = 'custom'`)
 - Erreurs génériques (500 – serveur)
- Peut aussi gérer **routes non trouvées** (404).
- Compatible avec **routes synchrones et asynchrones** (`async/await`).

5. Le middleware du router :

- Permet de **regrouper et organiser les routes** dans des modules/fichiers séparés.
- Chaque router agit comme une **mini-app** pour gérer un ensemble spécifique de routes (ex : produits, catégories).
- Permet de **centraliser la gestion des routes** et de **préfixer automatiquement les chemins** dans l'application principale.

Exemple 1 : Module router (routes/produits.js)

```

const express = require('express');
const router = express.Router();
// Middleware spécifique au router
router.use((req, res, next) => {
  console.log('Middleware spécifique au router des produits');
  next();
});
// Route principale pour les produits
router.get('/', (req, res) => {
  res.send('Liste des produits');
});
// Route pour un produit spécifique
router.get('/:id', (req, res) => {
  res.send(`Détails du produit avec ID : ${req.params.id}`);
});

```

```
module.exports = router;
```

Exemple 2 : Application principale (app.js)

```
const express = require('express');
const app = express();
const productRoutes = require('./routes/products');
// Middleware global pour toutes les routes
app.use((req, res, next) => {
  console.log('Middleware global pour toutes les routes');
  next();
});
// Attacher le router des produits avec préfixe '/products'
app.use('/products', productRoutes);

app.listen(3000, () => {
  console.log('Serveur démarré sur http://localhost:3000');
});
```

Points clés

- **router.use()** : Middleware spécifique à ce router.
- **app.use('/prefix', router)** : Préfixe automatique pour toutes les routes du module.
- Améliore la **lisibilité** et la **maintenance** du code.
- Permet de séparer les middlewares globaux et les middlewares spécifiques aux modules.

Communication entre les middlewares

Principe

- Une application Express peut utiliser **plusieurs middlewares** en même temps.
- Les middlewares sont exécutés **dans l'ordre où ils sont déclarés**.
- Chaque middleware peut **interagir avec req et res**, et passer la main au suivant avec **next()**.
- Si **next()** n'est pas appelé, la chaîne s'arrête et la requête reste bloquée.

Exemple combiné : morgan + favicon

1. Installer le middleware favicon :

```
npm install serve-favicon
```

2. Exemple de code :

```
const express = require('express');
const morgan = require('morgan');
const favicon = require('serve-favicon');
const path = require('path');
const app = express();
```

```
// Middleware pour journaliser les requêtes HTTP
app.use(morgan('dev'));
// Middleware pour servir la favicon
app.use(favicon(path.join(__dirname, 'favicon.ico')));
// Route principale
app.get('/', (req, res) => {
  res.send('Hello, World!');
});
app.listen(3000, () => {
  console.log('Serveur démarré sur http://localhost:3000');
});
```

Points clés

- **Ordre d'exécution** : morgan s'exécute avant favicon, car il est déclaré en premier.
- Chaque middleware peut **modifier la requête ou la réponse**, puis passer la main au suivant avec `next()`.
- Permet de **combiner des middlewares globaux et spécifiques** pour gérer les requêtes de manière flexible.

Chapitre 4 : NodeJs et les bases de données

Types de bases de données :

SQL (relationnelles)	NoSQL (non relationnelles)
MySQL	MongoDB
MariaDB	
PostgreSQL	
ORM (Object-Relational Mapping) : Sequelize	ODM (Object-Document Mapping) : Mongoose

Informations complémentaires :

- **SQL / relationnelles** : Les données sont organisées en **tables avec lignes et colonnes**, avec relations entre les tables.
 - Serveurs / SGBD : MySQL, MariaDB, PostgreSQL, Oracle, SQL Server.
 - Manipulation via **SQL**.
- **NoSQL / non relationnelles** : Les données sont organisées de façon **flexible** (documents, clés/valeurs, graphes).
 - Serveurs / SGBD : MongoDB, CouchDB, Redis, Cassandra.
 - Manipulation via **ODM** ou API spécifiques.

Outils :

- **phpMyAdmin** : Interface web pour gérer des bases de données MySQL/MariaDB. Sert de client SQL.
- **XAMPP** : Pack pour créer un environnement de développement local. Inclut PHP, MySQL et Apache pour développer des applications web.

Rappel :

- **MySQL** : Système de gestion de bases de données relationnelles (SGBD-R) open source, fonctionnant sur une architecture client-serveur.
- **SGBD-R** : Logiciel ou service pour stocker, mettre à jour et accéder aux données organisées en **tables**

Pourquoi base de données relationnelles ?

Résumé :

- Sans relations : certaines informations (ex. "Désignation service") se répètent pour chaque employé → **redondance**.
- Avec relations : création de tables liées par **clé étrangère**.
 - **Table Employés** : contient les données des employés + **ID_Service**.
 - **Table Services** : contient les informations des services.
- **Avantage** : éviter la répétition de données et faciliter la gestion.

Pourquoi client-serveur ?

MySQL fonctionne sur un **modèle client-serveur** dans un environnement réseau : le serveur MySQL gère les bases de données et traite les requêtes, tandis que les clients se connectent pour interagir avec les données.

- **Clients textuels et graphiques** :
 - MySQL (ligne de commande)
 - MySQL Workbench, phpMyAdmin (interfaces graphiques)
- **Clients pour langages de programmation** :
 - **PHP** : MySQLi, PDO
 - **Java** : JDBC
 - **Node.js** : **mysql**, **mysql2** (plus performant)
 - **Python** : MySQLdb, PyMySQL, mysql-connector-python
 - **Ruby** : Ruby-MySQL

Avantage : centralise la gestion des données tout en permettant à différents programmes et interfaces de communiquer avec le serveur pour accéder et manipuler les bases de données.

Communication client-serveur MySQL :

Le client envoie des **requêtes SQL** au serveur MySQL, qui les traite et renvoie les résultats :

- Client → Serveur MySQL

Le **SQL** utilisé pour communiquer entre le client et le serveur comprend plusieurs catégories:

- **LDD (Langage de Définition de Données)** : créer, modifier ou supprimer des tables et structures (ex : **CREATE TABLE**, **ALTER TABLE**).
- **LMD (Langage de Manipulation de Données)** : ajouter, modifier ou supprimer des données (ex : **INSERT**, **UPDATE**, **DELETE**).
- **LRD (Langage de Requête de Données)** : interroger les données (ex : **SELECT**).
- **LCT (Langage de Contrôle de Transactions)** : gérer les transactions (ex : **COMMIT**, **ROLLBACK**).

- **LCD (Langage de Contrôle de Données)** : gérer les permissions et accès aux données (ex : **GRANT**, **REVOKE**).

MySQL et Node.js

1. Installation et préparation

- Installer un serveur web local comme **XAMPP** (inclut MySQL/MariaDB et PhpMyAdmin).
- Installer le module **MySQL** pour Node.js et le middleware **express-myconnection** :
`npm install mysql express-myconnection`
- **XAMPP** permet de gérer la base de données via PhpMyAdmin.
- Composants principaux de XAMPP : serveur Apache, base de données MariaDB, PhpMyAdmin, etc.

2. Configuration dans Node.js

- Importer les modules :

```
const myConnection = require('express-myconnection');
```

```
const mysql = require('mysql');
```

- Définir les options de connexion à la base de données :

```
const optionBd = {
```

```
  host: 'localhost',
```

```
  user: 'root',
```

```
  password: '',
```

```
  port: 3306,
```

```
  database: 'notes_bd'
```

```
};
```

3. Middleware de connexion

- Utiliser le middleware pour gérer la connexion à la base :
`app.use(myConnection(mysql, optionBd, 'pool'));`
- **Remarque** : l'utilisation d'un **pool de connexions** améliore les performances, réduit la charge sur la base et accélère le traitement des requêtes fréquentes.

Récupérer les données se trouvant dans la BDD MySQL :

- Lorsque le middleware **express-myconnection** est utilisé, il ajoute la méthode **getConnection** à l'objet **req**.
- Cette méthode permet d'obtenir une connexion à la base pour exécuter des requêtes SQL.

Exemple complet :


```

app.get('/', (req, res) => {
  // Obtenir la connexion MySQL
  req.getConnection((err, myConnection) => {
    if (err) {
      console.log(err); // Gestion d'erreur si connexion échoue
    } else {
      // Exécuter la requête SQL
      myConnection.query('SELECT * FROM notes', [], (err, resultat) => {
        if (err) {
          console.log(err); // Gestion d'erreur si requête échoue
        } else {
          // Renvoyer les résultats vers la vue (template EJS)
          res.status(200).render('index', { resultat });
        }
      });
    }
  });
});

```

Explications :

1. `req.getConnection(...)` : récupère une connexion depuis le pool.
2. `myConnection.query(...)` : exécute une requête SQL (`SELECT * FROM notes`).
3. `res.render('index', { resultat })` : transmet les données récupérées à la vue EJS pour affichage.

Cela permet de lier directement les données MySQL à votre interface web.

Ajouter des données envoyées à travers un formulaire dans la BDD MySQL :

- Utiliser le middleware `express.json()` et/ou `express.urlencoded()` pour parser les données du formulaire (`req.body`).
- Utiliser `req.getConnection()` pour obtenir une connexion à la BDD et exécuter la requête SQL d'insertion.

Exemple complet :

```

app.post("/notes", (req, res) => {
  req.getConnection((err, myConnection) => {
    if (err) {
      console.log(err); // Gestion d'erreur si la connexion échoue
    } else {
      // Récupération des données du formulaire
      let titre = req.body.titre;
      let desc = req.body.description;

      // Requête d'insertion
      myConnection.query(
        "INSERT INTO notes (id, titre, description) VALUES (?, ?, ?)",

```

```

    [null, titre, desc],
    (err, resultat) => {
      if (err) {
        console.log(err); // Gestion d'erreur si l'insertion échoue
      } else {
        // Redirection vers la page d'accueil après insertion
        res.status(200).redirect('/');
      }
    }
  );
}
});
});

```

Points clés :

1. `req.body.titre` et `req.body.description` récupèrent les valeurs envoyées par le formulaire.
2. `INSERT INTO notes ...` ajoute une nouvelle ligne dans la table `notes`.
3. `res.redirect('/')` renvoie l'utilisateur vers la page d'accueil après l'insertion.

Cela permet de relier un formulaire HTML directement à votre base de données MySQL.

Communiquer avec MariaDB à partir de NodeJS à travers un ORM :

- **ORM (Object Relational Mapping)** : permet de manipuler la base de données via des objets JavaScript, sans écrire de SQL.
- **Abstraction** : masque la base de données réelle ; le code reste identique même si on change de SGBD.
- **Requêtes simplifiées** : méthodes clé en main comme `findAll`, `findOne`, `create()`, etc.
- **Exemple** :
 - SQL brut : `SELECT * FROM Users WHERE id = 4`
 - Avec ORM : `Users.find(4).then(user => {...}).catch(err => {...})`
- **Flux** : Application JS → ORM → Base de données → ORM → Application JS.

L'ORM sert de **couche intermédiaire** entre le code et la base, facilitant la gestion des données et réduisant l'écriture de SQL.

ORM Sequelize :

- **Sequelize** : ORM pour Node.js, basé sur les **promesses**, facilite la gestion asynchrone et le travail avec les bases SQL (MySQL, MariaDB, PostgreSQL, SQLite, SQL Server...).
- **Installation** : `npm install sequelize`

```
npm install mariadb --save # driver pour MariaDB
```

Le driver permet à Sequelize de réellement se connecter à la base MariaDB.

- **Configuration et connexion :**

1. **Importer Sequelize :** `const { Sequelize, DataTypes } = require('sequelize');`
2. **Créer une instance de connexion :**

```
const sequelize = new Sequelize(
  'my_bdd', // Nom de la BDD
  'root',   // Utilisateur
  '',       // Mot de passe
  {
    host: 'localhost',
    dialect: 'mariadb',
    dialectOptions: { timezone: 'Etc/GMT-2' },
    logging: false // Désactive les logs SQL
  }
);
```

3. **Tester la connexion :**

```
sequelize.authenticate()
  .then(() => console.log('Connexion à la BDD réussie !'))
  .catch(error => console.log('Impossible de se connecter : ${error}'));
```

- **Remarques :**

- **dialect** : indique le type de base (MariaDB ici).
- **logging: false** et **dialectOptions** évitent les messages d'avertissement dans la console.

Sequelize fournit ainsi une **couche d'abstraction** entre Node.js et la BDD, permettant d'utiliser JavaScript pour gérer les tables et données sans écrire de SQL brut.

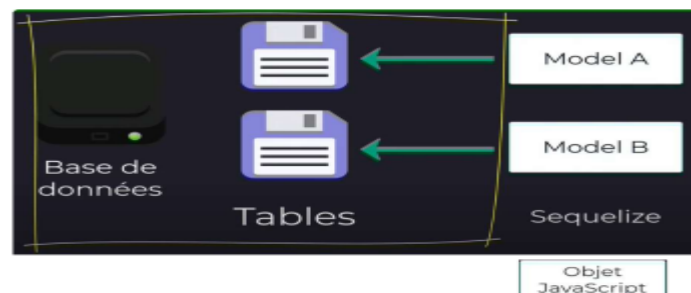
Présentation des modèles Sequelize :

Qu'est-ce qu'un modèle ?

- **Définition** : Un **modèle** est une abstraction représentant une **table** dans la base de données.
- **Fonction** : C'est un objet JavaScript fourni par Sequelize, configuré pour correspondre à la structure de la table (colonnes, types, contraintes...).
- **Rôle** : Il permet de manipuler les données de la table via JavaScript sans écrire de SQL brut.

Schéma conceptuel :

Chaque modèle correspond donc à une table et contient toutes les propriétés nécessaires pour que Sequelize puisse gérer les opérations



CRUD (Create, Read, Update, Delete) sur cette table.

Créer un modèle Sequelize :

- `sequelize.define('Etudiant', { ... }, { ... });`
- `'Etudiant'` : nom du modèle → correspondra à une table `Etudiants` dans la BDD.
- Objet `{ id, name }` : définit les **colonnes** de la table avec leurs types et contraintes.

Exemple de colonnes :

`id: {`

`type: DataTypes.INTEGER,`

`primaryKey: true,`

`autoIncrement: true`

`},`

`name: {`

`type: DataTypes.STRING,`

`allowNull: false}`

Options du modèle :

- `timestamps: true` → Sequelize gère automatiquement les champs de création et modification.
- `createdAt: 'created'` → nom personnalisé pour le champ de création.
- `updatedAt: false` → pas de champ de mise à jour.

Explication :

- `sequelize.define()` crée le modèle et configure la table dans la BDD.
- Les modèles permettent de manipuler les données en JS sans écrire de SQL.

Synchroniser le modèle avec la BDD :

- **Objectif** : créer les tables correspondantes aux modèles définis dans la base de données.
- **Code** :

```
const { Sequelize, DataTypes } = require('sequelize');
const EtudiantModel = require('./models/etudiant.js');
```

```
let Etudiant = EtudiantModel(sequelize, DataTypes);
```

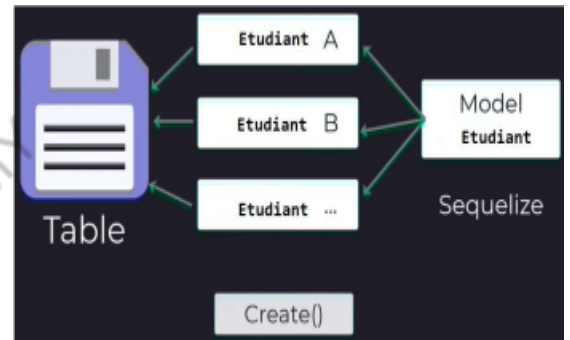
```
sequelize.sync({ force: true })
  .then(_ => {
```

```
console.log('La BDD a été bien synchronisée');
});
```

- **Explications :**
 - `EtudiantModel(sequelize, DataTypes)` : instancie le modèle dans l'application.
 - `sequelize.sync()` : crée la table si elle n'existe pas.
 - Option `force: true` : **supprime et recrée** la table à chaque exécution (utile pour tests, attention en prod).
 - La console affiche un message lorsque la synchronisation est réussie.
- **Remarque :** sans `force: true`, `sync()` ne fera que créer les tables manquantes sans supprimer les existantes.

Instancier un modèle Sequelize (ajouter une ligne à la table) :

- **Objectif :** ajouter une ligne dans la table via le modèle Sequelize.
- **Principe :** chaque modèle possède la méthode `create()` qui insère une nouvelle entrée dans la table.
- **Explication :**
 - On manipule **des objets JavaScript** (Étudiant A, Étudiant B) via le modèle.
 - `create()` traduit cet objet en **requête SQL INSERT** vers la base de données.



Ajouter des lignes à la table :

- **Objectif :** insérer tous les éléments d'un tableau dans la table via le modèle.
- **Principe :** après la synchronisation de la BDD (`sequelize.sync()`), on parcourt le tableau `étudiants` et on appelle `Etudiant.create()` pour chaque élément.
- **Exécution :**

```
étudiants.map(étudiant => {
  Etudiant.create({ id: étudiant.id, name: étudiant.name })
    .then(s => console.log(s.toJSON()));
});
```

- **Effet :** chaque objet du tableau devient une **nouvelle ligne** dans la table `Etudiant`, et `toJSON()` permet d'afficher l'entrée créée dans la console.

Les méthodes couramment utilisées dans Sequelize :

- `model.create()` : Crée une nouvelle ligne dans la table.
- `model.findAll()` : Récupère **toutes** les lignes de la table.
- `model.findOne()` : Récupère la **première ligne** correspondant à des critères spécifiques.
- `model.update()` : Met à jour **une ou plusieurs lignes** existantes.
- `model.destroy()` : Supprime **une ou plusieurs lignes** de la table.

Ces méthodes permettent de gérer facilement les données d'une table via le modèle Sequelize, sans écrire de SQL brut.

MongoDB

- **Type de base de données** : Orientée documents (NoSQL).
- **Organisation** : Les données sont stockées dans des **collections** (équivalent des tables) composées de **documents** (équivalent des lignes).
- **Document** : Objet JSON stocké en BSON (binaire).
 - Exemple :

```
{ "_id": "5ae492a8c0adb22dc80ab54", "nom": "iPhone X", "prix": 1159 }
```

Schéma flexible : Une collection peut contenir des documents de structure différente.

- Exemple :

```
{ "_id": "5ae492a8c0adb22dc80ab54", "nom": "iPhone X", "prix": 1159 }
```

```
{ "_id": "4afe3f683611502135847759", "nom": "iPhone X", "description": "Dernier iPhone" }
```

MongoDB vs DB relationnelle :

- **Relations** :
 - **MongoDB** : pas de clés étrangères ni de jointures automatiques.
 - On peut **imbriquer un document dans un autre**, mais il ne peut pas référencer directement un autre document.
 - Les références via **id** sont possibles, mais doivent être gérées manuellement.
 - **Bases relationnelles** : utilisent des **clés étrangères** pour lier les tables automatiquement.
- **Type** : MongoDB est une base **NoSQL**, tandis que MySQL, PostgreSQL, etc., sont relationnelles (SQL).

MongoDB et NodeJS

- Pour interagir avec **MongoDB** depuis **Node.js**, il faut utiliser un **driver** (module qui fait le lien entre Node.js et la base).
- Le module officiel pour Node.js s'appelle **mongodb**.
- Installation via npm : `npm install mongodb --save`

Communiquer avec MongoDB à partir de NodeJS : "mongoose"

- **Mongoose** est un **ODM** (Object Document Mapping) pour MongoDB.
- Il permet de créer un **modèle** pour chaque collection, représentant les documents sous forme d'objets JavaScript.
- Le modèle fournit des **méthodes prédéfinies** pour manipuler la collection (ajout, lecture, modification, suppression) sans écrire de requêtes MongoDB directes.
- Installation : `npm install mongoose --save`